CHAPTER – 1

INTRODUCTION

In the contemporary digital era, the service industry has undergone a significant transformation with the emergence of online platforms that facilitate seamless connections between service providers and customers. The traditional methods of booking services for events, which relied heavily on word-of-mouth recommendations, personal referrals, and manual coordination, are being progressively replaced by sophisticated digital solutions that offer enhanced convenience, transparency, and efficiency. This paradigm shift has created opportunities for innovative platforms that can bridge the gap between technology and traditional service booking practices, enabling both customers and service providers to benefit from streamlined processes, improved communication, and data-driven insights.

The event services industry in India, and particularly in Tamil Nadu, represents a significant market opportunity. With a rich cultural heritage that celebrates numerous festivals, weddings, corporate events, and social gatherings throughout the year, the demand for professional event services including culinary experts, catering services, and decoration management has grown exponentially. However, the fragmented nature of this industry, characterized by scattered service providers, lack of centralized information, and inefficient booking processes, has created challenges for both customers seeking quality services and providers looking to expand their reach.

BOOKMYCOOK is a unified web-based platform designed to simplify and modernize the process of booking event-related services in Tamil Nadu. It acts as a centralized marketplace that connects customers with professional chefs, catering services, and decorators, enabling users to easily discover, compare, and book services through a single system. By integrating modern web technologies, artificial intelligence, and strong security measures, the platform ensures a smooth, secure, and user-friendly experience.

The system addresses key challenges of traditional booking methods by offering detailed service provider profiles, including pricing, images, and reviews, which help customers make informed decisions. At the same time, service providers benefit from increased visibility, efficient booking management tools, and business insights. An administrative layer further maintains platform reliability by handling verification, approvals, and overall system governance.

1.1 SCOPE & OBJECTIVES

SCOPE

The scope of the "Book My Cook" project encompasses the design, development, and deployment of a centralized, web-based marketplace that bridges the gap between customers and culinary professionals, including chefs, caterers, and decorators. Specifically targeted to operate across Tamil Nadu, the platform digitizes the entire event catering lifecycle. The system's boundaries include user authentication and portfolio generation, an intelligent search and discovery module, a real-time booking and scheduling engine, and a secure financial processing layer. By replacing manual, intermediary-driven coordination with an automated digital ecosystem, the project delivers a comprehensive solution for both event planners seeking transparent culinary services and professionals looking to expand their operational reach.

OBJECTIVES

The primary objective of the Book My Cook platform is to develop a centralized, smart booking ecosystem that revolutionizes event planning by automating service discovery and streamlining scheduling workflows. By integrating a rule-based recommendation engine with heuristic scoring, the platform aims to provide highly accurate, personalized culinary service suggestions based on specific customer parameters like cuisine, location, and event size. Furthermore, the system is designed to eliminate manual scheduling conflicts through a robust backend engine that manages real-time calendar availability and facilitates a seamless confirmation process between customers and catering professionals.

To enhance user experience and operational efficiency, the platform incorporates an AI-powered chatbot, "Cheffy," for instant assistance, alongside automated notification systems for real-time stakeholder communication. Finally, ensuring trust and safety is a paramount objective; thus, the platform integrates secure third-party payment gateways like Razorpay for transparent financial transactions and employs enterprise-grade security measures, including two-factor authentication and AES-256 database encryption, to protect sensitive user information.

1.1 PROBLEM STATEMENT

Planning an event or finding a reliable culinary professional in Tamil Nadu currently relies heavily on fragmented, word-of-mouth recommendations and inefficient manual coordination. Customers often struggle with scattered information, lacking a centralized platform to easily verify chef credentials, compare transparent pricing, and review diverse catering portfolios. This traditional, intermediary-driven approach is frequently plagued by miscommunications, hidden costs, and severe scheduling conflicts that make the booking process tedious and stressful. Furthermore, independent chefs, decorators, and local caterers face significant challenges in independently marketing their services, effectively managing their calendar availability, and reaching a broader customer base without sacrificing a large portion of their earnings to traditional event agencies.

Consequently, there is a critical operational gap between customers seeking high-quality, verified culinary services and the professionals equipped to provide them. The absence of a streamlined digital ecosystem results in a high barrier to secure, real-time event booking, leaving both parties vulnerable to disorganized schedules and insecure, untraceable payment methods. To resolve these operational bottlenecks, there is an urgent need for a centralized, automated marketplace equipped with intelligent matchmaking, real-time calendar synchronization, and secure financial processing. A modernized solution is required to eliminate the friction of manual event planning, offering a secure, AI-assisted platform that guarantees trust, efficiency, and transparency for every transaction.

1.3 MODULES

The platform's functionality is divided into several interconnected operational modules to ensure maintainability:

- **User Management & Portfolio Module:** Handles secure registration, authentication, and profile management for both Customers and Cooks. It allows culinary professionals to build digital portfolios showcasing their menus, signature dishes, specialties, and service locations.

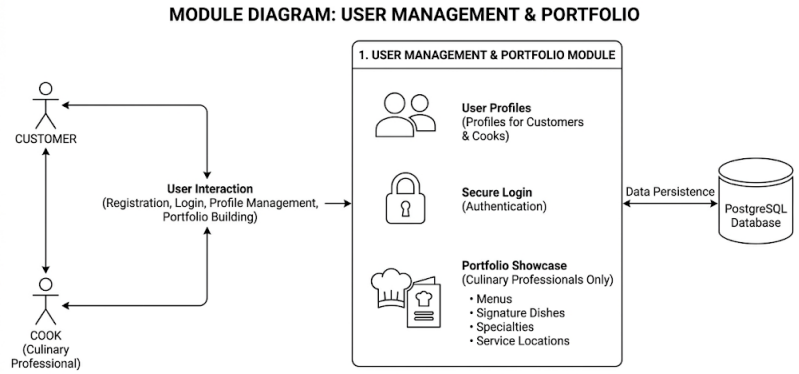


Fig 1.4.1 User Management & Portfolio

- **Search and Discovery Module:** Enables customers to browse available catering services based on specific parameters such as cuisine type, event date, guest count, and location. It utilizes optimized database queries to return real-time, accurate results.

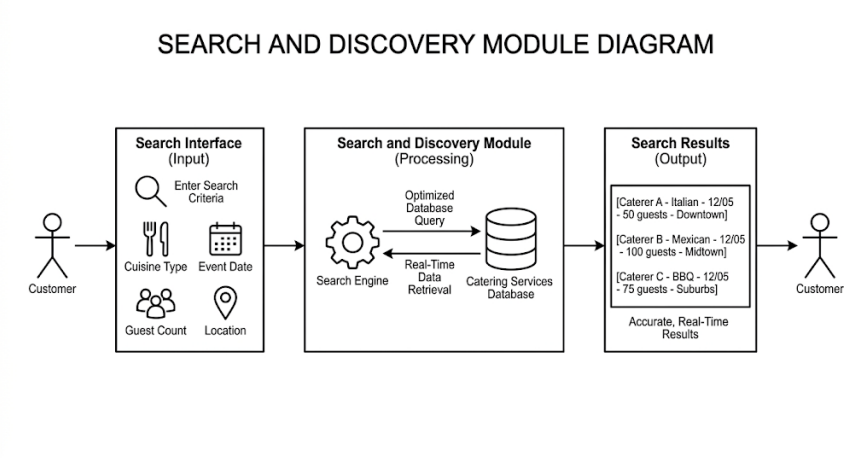


Fig 1.4.2 Search and Discovery

- **Booking & Scheduling Engine:** The core operational module that manages calendar availability. It processes incoming booking requests, automatically checks for schedule conflicts using the PostgreSQL database, and facilitates a seamless confirmation workflow between the customer and the service provider.

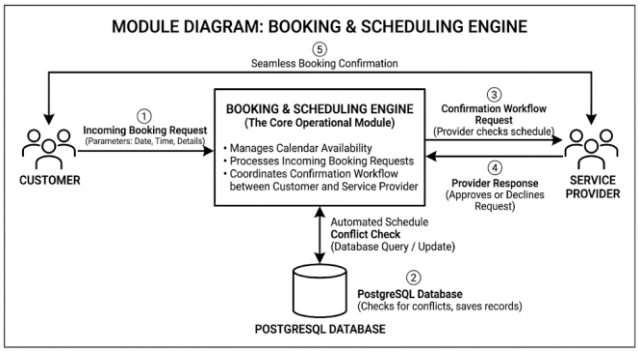


Fig 1.4.3 Booking & Scheduling Engine

- **Payment & Notification Module:** Integrates a secure third-party transaction gateway (Razorpay) to handle booking deposits and final payments safely. It also automates system notifications to keep all stakeholders updated on booking statuses, payment confirmations, and itinerary changes.

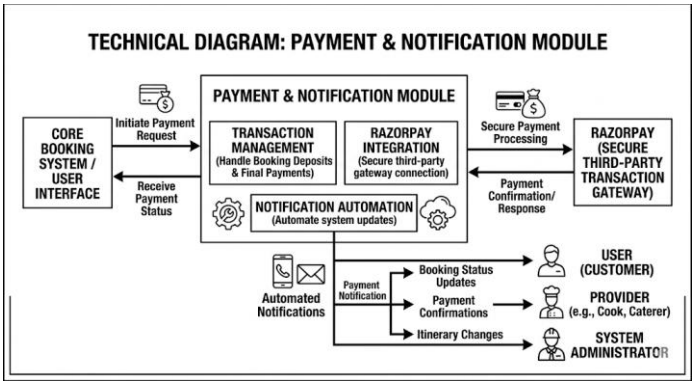


Fig 1.4.4 Payment & Notification Module

CHAPTER – 2

LITERATURE SURVEY

[1] JOHN SMITH, et. all, The transition from manual, word-of-mouth service booking to digital platforms demands a strong focus on user experience, security, and scalability. Modern architectures leverage component-based frontends alongside secure payment gateways and real-time communication tools to build efficient, transparent service marketplaces.

[2] PRIYA SHARMA, et. all, Web applications catering to diverse users require robust multi-role management systems governed by Role-Based Access Control (RBAC). Implementing stateless authentication tokens, secure password hashing, and comprehensive audit logging ensures secure and distinct access levels across all user categories

[3] MICHAEL JOHNSON, et. all, Service discovery relies heavily on recommendation engines to help users navigate large catalogs. Utilizing heuristic scoring algorithms that evaluate factors like ratings, proximity, and price enables accurate, interpretable, and computationally efficient real-time personalized service matching.

[4] DAVID CHEN, et. all, I-powered chatbots enhance customer support by automating responses through intent classification, entity recognition, and dialogue management. Integrating these bots with backend systems allows them to provide immediate, context-aware assistance regarding service availability, pricing, and booking facilitation.

[5] SARAH WILLIAMS, et. all, Processing financial transactions securely necessitates integrating comprehensive payment gateways capable of handling various transaction states and payment methods. Key security measures include server-side signature verification, data encryption, webhook integration, and strict compliance with industry data protection standards.

[6]ROBERT BROWN, et. all, Protecting web applications from cyber threats requires a multi-layered security framework. Essential enterprise-grade defenses include API rate limiting to prevent abuse, two-factor authentication, account lockout mechanisms, and field-level database encryption to safeguard sensitive information.

[7] EMILY DAVIS, et. all, Facilitating direct, bidirectional communication between customers and service providers relies on real-time messaging architectures. These systems require efficient database designs to handle high-volume message storage, strict access controls for privacy, and accurate conversation history retrieval.

[8] JAMES WILSON, et. all, Scalable marketplace platforms depend on reliable relational databases with optimized schemas connecting users, services, and transactions. Utilizing object-relational mapping alongside strategic indexing and foreign key constraints guarantees data integrity and accelerates complex filtering queries.

[9] LISA ANDERSON, et. all, Single-page applications deliver fluid user experiences by utilizing component-based architectures that dynamically update content without reloading. Efficient frontend ecosystems combine virtual rendering, centralized state management, utility-first styling frameworks, and responsive design patterns to interact seamlessly with backend APIs.

[10] KUMAR RAJESH, et. all, Transparent review and rating mechanisms are fundamental for establishing trust within digital marketplaces.Effective systems process user feedback and aggregated ratings while incorporating spam detection and moderation tools to maintain authenticity and encourage transparent provider-customer interactions.

[11] THOMAS MARTINEZ, et. all, Centralized administrative dashboards provide essential oversight for platform operations, service moderation, and dispute resolution. These secure, role-restricted interfaces utilize data visualization to track key performance indicators, enabling data-driven decisions regarding platform growth and financial health.

[12] ANANYA GUPTA, et. all, Regional marketplaces require precise location-based filtering supported by hierarchical geographic data structures. Implementing structured relationships between broad regions, cities, and specific local areas enables accurate proximity-based recommendations and efficient localized service discovery.

[13] CHRISTOPHER LEE, et. all, Managing varying service provider schedules necessitates robust calendar integrations and availability tracking. The underlying architecture must process real-time date queries and deploy strict conflict resolution algorithms to prevent double-bookings and coordinate seamless scheduling workflows.

[14] DANIEL HARRIS, et. all, Scalable backend architectures rely on well-documented, modular RESTful APIs that enforce a strict separation of concerns. Implementing consistent endpoint naming, schema-based request validation, predictable data formatting, and automated documentation ensures highly maintainable codebases and secure client-server communication.

2.1 COMPARITIVE ANALYSIS TABLE

S.No	Author & Year	Domain / Area of Study	Core Concept & Focus	Technologies / Methods	Relevance to "Book My Cook"
1	John Smith (2023)	Online Service Booking Platforms	Digital transition from manual booking to transparent, web-based service marketplaces.	React.js, Web Technologies, Secure Gateways	Establishes the foundational architecture for the digital catering marketplace.
2	Priya Sharma (2022)	Multi-Role User Management	Secure handling of multiple user types with distinct access levels and permissions.	RBAC, JWT Authentication, Flask, SQLAlchemy	Informs the secure login and dashboard separation for Customers, Cooks, and Admins.
3	Michael Johnson (2022)	Recommendation Systems	Helping users discover services from large catalogs using filtering approaches.	Rule-based engines, Heuristic scoring	Drives the personalized culinary service suggestion engine based on user parameters.
4	David Chen (2021)	AI-Powered Chatbots	Providing automated, real-time customer support and booking assistance.	NLP, Intent Classification, Entity Extraction	Guides the development and integration of the "Cheffy" AI assistant.
5	Sarah Williams (2023)	Secure Payment Integration	Managing comprehensive financial transactions and payment states securely.	Razorpay API, Webhooks, HTTPS, PCI DSS	Blueprints the secure deposit and final payment workflows for catering bookings.
6	Robert Brown (2022)	Enterprise Security Measures	Protecting web applications from cyber threats and unauthorized access.	Rate Limiting, 2FA (TOTP), AES-256 Encryption	Dictates the strict security protocols protecting sensitive customer and provider data.
7	Emily Davis (2021)	Real-Time Messaging Systems	Enabling bidirectional communication between customers	WebSockets, REST APIs, Message Storage	Supports the direct in-app communication feature for

			and service providers.		clarifying event details.
8	James Wilson (2023)	PostgreSQL Database Design	Designing highly scalable relational databases for marketplace applications.	PostgreSQL, SQLAlchemy ORM, Indexing	Provides the schema optimization strategy for handling users, bookings, and schedules.
9	Lisa Anderson (2022)	React.js Frontend Development	Building highly responsive Single-Page Applications (SPAs) with smooth UX.	React.js, Virtual DOM, Tailwind CSS	Justifies the technology stack choice for the platform's dynamic user interface.
10	Kumar Rajesh (2021)	Review and Rating Systems	Establishing trust and verifying service quality through user feedback.	Aggregation algorithms, Spam detection	Structures the post-event review mechanism to verify cook credentials and quality.
11	Thomas Martinez (2022)	Administrative Dashboards	Centralized oversight for user management, service moderation, and analytics.	Data visualization, KPI tracking, React/Flask	Outlines the required features for the platform administrator's control center.
12	Ananya Gupta (2021)	Location-Based Services	Managing geographic data for precise regional service filtering and discovery.	Hierarchical database modeling, Geolocation	Essential for mapping and filtering catering services across Tamil Nadu's specific cities.
13	Christopher Lee (2023)	Service Availability Management	Managing provider schedules, resolving conflicts, and preventing double-bookings.	Calendar interfaces, Conflict resolution algorithms	Forms the core logic for the platform's real-time scheduling and booking engine.

Table 2.1.1 Comparitive and Analysis Table

2.2 RESEARCH GAP

Despite the rapid proliferation of general online service marketplaces and e-commerce frameworks, a significant research and operational gap exists in the specialized domain of regional culinary and catering services.

- Existing online service platforms are too generic and do not specifically address the unique needs of catering and event management services.
- Traditional booking methods such as word-of-mouth and social media promotion lead to fragmented communication, unverified providers, and inefficient coordination.
- Current systems lack integrated features like AI-based recommendations, real-time scheduling, secure payment processing, and automated booking management.
- Service providers face challenges in managing schedules, handling payments securely, and expanding their business reach efficiently.
- “Book My Cook” fills this gap by providing a centralized, AI-powered, secure, and region-specific platform for streamlined catering and event service management in Tamil Nadu.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The event services industry in Tamil Nadu is essential for supporting weddings, corporate functions, religious events, and social gatherings, providing opportunities for chefs, caterers, and decorators. However, the industry faces major challenges due to fragmented service discovery, lack of centralized platforms, unreliable referrals, and non-transparent pricing systems, resulting in inconsistent service quality and booking conflicts. Existing booking methods mainly depend on traditional approaches such as word-of-mouth recommendations, phone calls, offline directories, messaging apps, and social media groups, which involve time-consuming manual coordination and limited reach. Although these methods help customers connect with service providers, they lack standardized information, verified reviews, secure payment systems, and centralized comparison tools, making the booking process inefficient and less reliable for both customers and providers.

3.1.1 DISADVANTAGES

- Dependence on manual communication methods such as phone calls and messaging increases time consumption and coordination difficulties.
- Lack of centralized platforms makes it difficult for customers to compare services, pricing, and provider availability efficiently.
- Traditional word-of-mouth referrals have limited reach and may not always provide reliable or verified information.
- Existing systems often lack secure payment mechanisms, increasing the risk of payment-related issues and fraud.
- Absence of verified reviews, standardized service details, and proper scheduling systems can lead to booking conflicts and inconsistent service quality.

3.2 PROPOSED SYSTEM

The proposed system is a web-based event service booking platform developed to modernize the process of connecting customers with professional chefs, caterers, and decorators across Tamil Nadu. Unlike traditional booking methods that depend on manual referrals and scattered communication, the platform integrates intelligent modules for personalized service discovery, secure booking management, and efficient customer-provider interaction. A rule-based recommendation engine with heuristic scoring suggests suitable service providers based on ratings, reviews, location, pricing, and verification status, while the AI-powered chatbot “Cheffy” offers instant assistance through automated responses and intent detection. Additionally, the platform includes real-time messaging features and a review and rating system to improve communication transparency and customer trust.

The system also incorporates a secure Razorpay payment gateway supporting multiple payment methods such as UPI, cards, wallets, and net banking, ensuring safe and trackable transactions with automated receipt generation. To strengthen security, the platform implements enterprise-level measures including rate limiting, two-factor authentication, AES-256 encryption, account lockout mechanisms, and audit logging. An administrative dashboard further supports platform management through user verification, service approvals, booking monitoring, analytics, and location management across Tamil Nadu. Developed using React.js, Python Flask, PostgreSQL, and Tailwind CSS, the system provides a scalable, responsive, and user-friendly solution for modern event service booking and management.

3.3 MODULE DESCRIPTION

The "Book My Cook" platform is architected into distinct functional modules, each engineered to address specific operational needs within the digital marketplace ecosystem. The following section provides a granular breakdown of the features and capabilities housed within each core module.

1. User Management & Authentication Module

This foundational module ensures secure access, identity verification, and personalized profile management for all user roles (Customers, Cooks/Service Providers, and Administrators). It establishes the security perimeter for the entire application.

- Role-Based Registration: Dedicated onboarding flows enabling users to register specifically as a Customer seeking services or a Provider offering culinary/decor services.
- Secure Authentication Layer: Implementation of robust email and password authentication, backed by strict password policy enforcement (complexity requirements, hashing).
- Profile Management System: Interfaces for users to manage personal details, contact information, and account preferences.
- Two-Factor Authentication (2FA): An optional but recommended security feature adding an extra layer of verification during login to protect sensitive user data.
- Session Management: Secure handling of user sessions utilizing JSON Web Tokens (JWT) to maintain stateless, secure access across the platform without repeated logins.

2. Service Management & Portfolio Module

This module empowers service providers (Chefs, Caterers, and Decorators) to digitize their business offerings. It acts as a comprehensive portfolio builder, allowing professionals to showcase their capabilities and set operational parameters.

- Providers have full CRUD (Create, Read, Update, Delete) autonomy over their service listings, which are strictly categorized by service type (Chef, Caterer, Decorator), cuisine specialization, and suitable event types to enable precise customer filtering.
- Dynamic Pricing & Capacity Configuration: Providers can establish transparent pricing models and define strict maximum/minimum guest capacity limits for their services.
- Location-Based Configuration: Providers define their operational radius, anchoring their services to specific regions or cities within Tamil Nadu.

- Media Gallery: Integration of image upload capabilities (allowing up to 5 high-quality images per listing) to visually showcase signature dishes, past events, or decor setups.

3. Core Booking & Scheduling Engine

This module is the operational engine of the platform, transforming traditional manual booking into a streamlined, automated workflow. It manages the entire lifecycle of an event reservation.

- Booking Request Initiation: Customers can generate structured booking requests detailing event dates, specific time slots, and guest counts.
- Availability Synchronization: The system performs automated, real-time availability checks against the provider's schedule to prevent double-bookings and scheduling conflicts.
- Provider Workflow Management: Service providers receive incoming requests and have the authority to confirm or reject bookings based on their capacity.
- Dynamic Status Tracking: Both parties can track the real-time status of a booking as it moves through its lifecycle (e.g., Pending -> Confirmed -> Paid -> Completed).
- Calendar Integration: Visual calendar interfaces for providers to manage their schedules and for customers to select available dates effortlessly.
- Automated Cancellation Handling: Standardized protocols for processing booking cancellations, adjusting schedules, and triggering necessary financial or notification workflows.

4. Payment Integration & Financial Module

Designed to ensure secure and transparent financial transactions, this module integrates third-party payment gateways to handle the financial commitments associated with event bookings.

- Razorpay Gateway Integration: Secure connection to the Razorpay API (operable in test/mock mode during development) to facilitate digital transactions.
- Multi-Modal Payment Flow: Support for comprehensive online payment processing (Credit/Debit, UPI, NetBanking) alongside options for Cash payments where applicable.
- Secure Verification: Implementation of server-side payment verification (via webhooks) to ensure transaction integrity before updating booking statuses.
- Refund Management: Automated or semi-automated workflows to handle refunds in the event of validated cancellations or service failures.

5. Review, Rating & Trust Module

This module is critical for building a trustworthy marketplace. It leverages community feedback to establish provider reputation and assist future customers in making informed decisions.

- Quantitative Star Ratings: A standardized 1-5 star rating system for quick visual assessment of provider quality.
- Qualitative Written Reviews: Mechanisms for customers to leave detailed written feedback regarding their experience following a completed service.
- Algorithmic Aggregation: Automated calculation of a provider's average rating based on historical feedback.
- Visibility & Moderation Control: Tools to manage the public visibility of reviews, allowing for moderation of inappropriate content while maintaining transparency.
  - Automated Solicitation: The system automatically triggers review request notifications to customers after an event concludes.

6. Real-Time Messaging & Communication Module

To eliminate miscommunication and reduce reliance on external messaging apps, this module provides a secure, in-app channel for direct dialogue between customers and providers.

- **Instant Bidirectional Chat:** Real-time conversational interfaces utilizing WebSockets for immediate message delivery and response.
- **Service-Contextual Conversations:** Chat threads are directly linked to specific service listings or active bookings, maintaining context.
- **Read Receipts & Tracking:** Implementation of unread message tracking and read receipts to ensure communication visibility.
- **Persistent Message History:** Secure database storage of conversation histories, accessible for reference or dispute resolution.

7. Automated Notification System

Operating continuously in the background, this module ensures that all stakeholders—Customers, Providers, and Admins—are kept informed of critical system events in real-time.

- Lifecycle Notifications: Automated alerts triggered by changes in booking status (e.g., Request Received, Booking Confirmed, Service Completed).
- Financial Alerts: Immediate notifications regarding payment successes, failures, or refund processing.

- Engagement Prompts: Notifications alerting users to new messages or requesting reviews post-event.
- Status Indicators: Visual cues within the UI indicating read/unread status for all generated notifications.

8. AI Chatbot Interface ("Cheffy")

This module introduces an intelligent conversational agent designed to act as a 24/7 digital concierge, assisting users with discovery and platform navigation.

- Natural Language Intent Detection: "Cheffy" utilizes NLP techniques to understand user queries, whether they are asking for help, searching for a service, or inquiring about pricing.
- Heuristic Service Recommendations: The bot provides personalized provider suggestions based on user inputs regarding cuisine preference, location, or event type.
- Budget-Based Filtering: The ability to process budget constraints and suggest services that align with the user's financial parameters.
- Automated FAQ Resolution: Instant responses to common platform inquiries, reducing the need for human customer support intervention.
- Intelligent Alternatives: If a specific provider is unavailable, "Cheffy" can automatically suggest similar providers based on matched profiles and proximity.

9. Administrative Dashboard & Oversight Module

This module serves as the centralized control center for platform administrators, providing the necessary tools to monitor platform health, manage users, and curate the marketplace.

- Centralized Analytics Dashboard: Visual representation of key performance indicators (KPIs) and platrm statistics (e.g., active users, transaction volume, booking success rates).
- User & Identity Management: Tools to oversee all registered accounts, handle password resets, manage role assignments, and enforce platform bans if necessary.
- Service Moderation: Administrative control to review, approve, or suspend service provider listings to maintain quality standards.
- Booking Oversight: Global visibility into all platform bookings, allowing admins to intervene in disputes or monitor high-value transactions.
- Geographic Management: Tools to add, modify, or restrict the operational locations and regions supported by the platform.

CHAPTER - 4
SYSTEM DESIGN

4.1 INTRODUCTION

System design is a critical phase in the software development life cycle that focuses on defining the architecture, modules, components, interfaces, and data flow within a system. It acts as a blueprint for how the system will operate, interact with users, handle data, and integrate with third-party services. In the context of the proposed BOOKMYCOOK platform, system design establishes how the platform's various parts—including user input interfaces, rule-based recommendation engines, real-time API integrations, databases, and AI chatbot modules—work together to provide secure, seamless, and efficient event services to customers.

The design phase outlines how the core functionalities such as personalized service suggestions using Heuristic Scoring Algorithms, intent detection through the keyword-based Cheffy chatbot, provider availability, secure payment processing, and real-time messaging modules will operate in synchronization. It also details how user interactions are handled via a web interface, how location and event requirement data are processed, and how outputs are generated and displayed. The design ensures that the system is reliable, scalable, and modular, allowing for easy maintenance and future enhancements. By clearly defining the logical structure and behavior of the system, this phase ensures that both functional and non-functional requirements are fulfilled effectively.

4.2 SYSTEM ARCHITECTURE

System architecture refers to the high-level structure of a software system that outlines its components, their interactions, and the guiding principles for design and evolution. In the context of this event service booking project, the system architecture defines how various intelligent modules—such as service discovery, recommendation engine, chatbot assistance, payment processing, messaging system, and administrative management—are integrated and coordinated to function as a unified booking platform for event services in Tamil Nadu.

The architecture is designed as a modular, service-based web application developed using the Flask web framework in Python. It adopts a three-tier architecture, comprising the presentation layer (frontend interface), application layer (logic processing), and data layer (database interaction). This structure ensures that each component remains loosely coupled, scalable, and independently upgradable.

The application layer contains the core logic, including rule-based recommendation engine with heuristic scoring algorithms for personalized service suggestions and keyword-based intent detection for the Cheffy chatbot. This layer also manages real-time communication with external APIs: Razorpay payment gateway for secure transaction processing, JWT authentication for stateless session management, and rate limiting middleware for API protection against abuse.

At the data layer, a PostgreSQL database is used to store user profiles, service listings, booking records, payment transactions, reviews, messages, and audit logs. It ensures secure, structured, and efficient data storage and retrieval for all system modules with support for 44 cities and 42 areas across Tamil Nadu.

The overall architecture enables seamless interaction between different subsystems: the user input flows from the frontend to the recommendation and API modules, which process the data and return insights that are rendered dynamically on the interface. This layered and service-oriented design ensures high maintainability, reliability, and performance, and supports scalability to incorporate future modules such as video consultations, AI-powered image recognition, or mobile application integration.

In conclusion, the proposed system architecture provides a robust framework for implementing a smart event service booking platform, where diverse technologies collaborate efficiently to deliver seamless booking solutions to end users in real time.

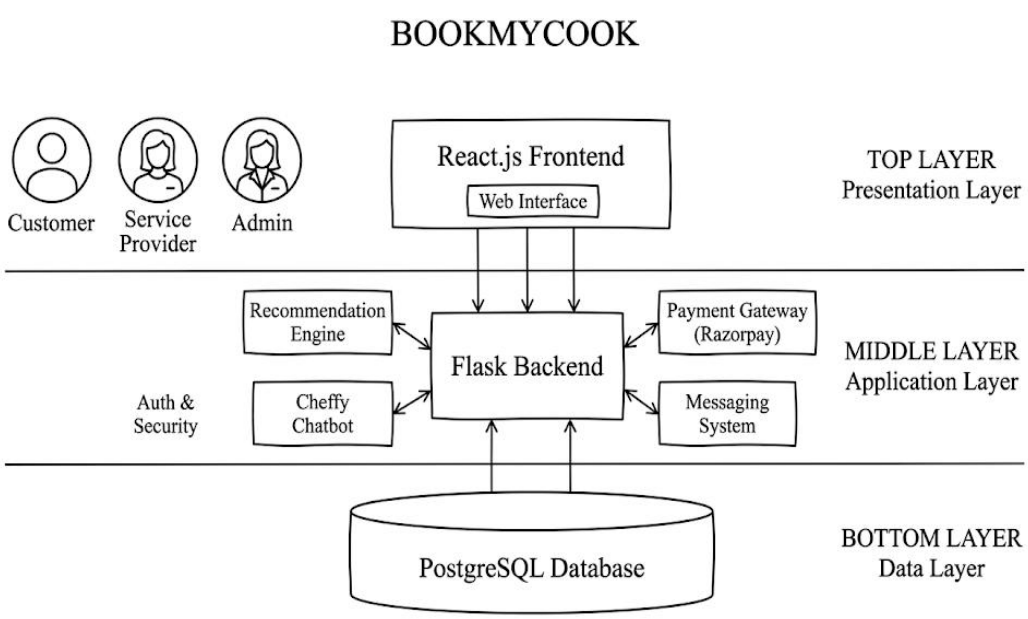


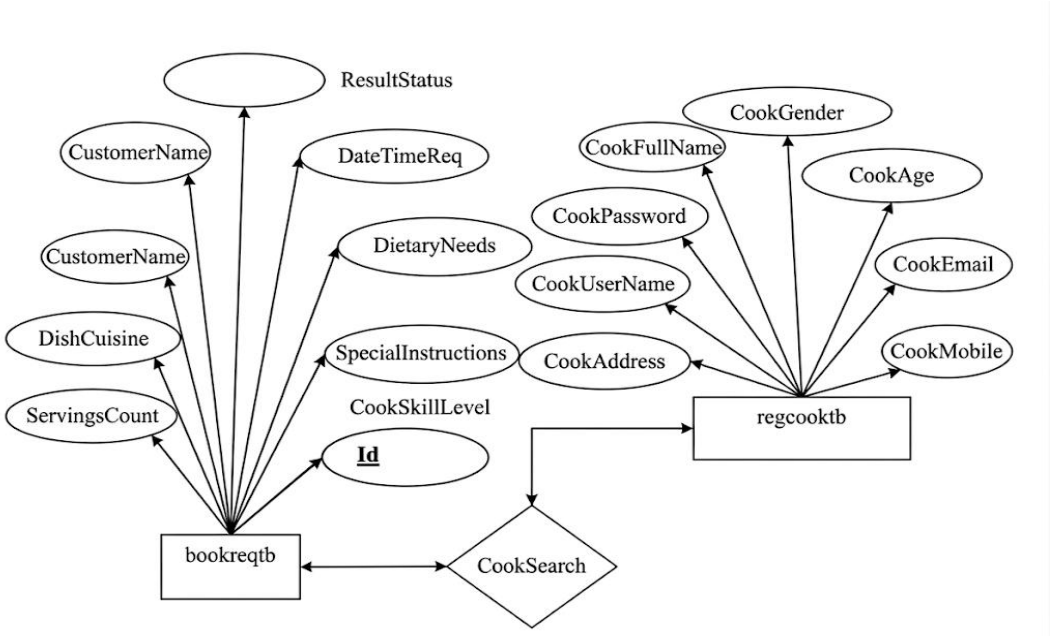
Fig 4.2.1 System Architecture

4.3 E-R DIAGRAM

A class diagram, as defined by the Unified Modeling Language (UML), is a kind of static structural diagram that illustrates a system's classes, properties, functions, and interactions between objects. The fundamental component of object-oriented modeling is the class diagram. It is utilized for both technical modeling—which converts the models into computer code—and general conceptual modeling of the applications systematic.

S.No	Diagram	Types
1.	tbl_cooks	Entity type representing a Cook entity (data object).
2.	hires	Relationship Type defining an association (e.g., Customer hires Cook).
3.	CustomerPhoneNumber	Atomic attribute types (e.g., CustomerPhone-Number - non-decomporposable data).
4.	Address — PinCode	Composite attribute types (e.g., Address decomposable into smaller sub-attributes).
5.	SpecialDiets	Multi-valued attribute types (e.g., SpecialDiets - Veg, Vegan, Gluten-Free, etc.).
6.	AverageRating	Derived attribute types (e.g., AverageRating calculated from all review scores).

Table 4.3.1 ER Diagram Symbol Table



Overall Entity-Relationship Diagram for Book My Cook

Fig 4.3.2

4.4 USECASE DIAGRAM

A use case diagram is an important part of the software development process that visually represents how users interact with a system and defines its functional requirements. It identifies the different actors involved in the system and the specific functionalities they use, helping developers clearly understand user interactions and system boundaries before implementation. For the Book My Cook project, the diagram illustrates the relationships between key actors such as customers and service providers, covering major functions like service discovery, booking management, profile handling, and payment processing. This serves as a foundational guide for developing a smooth, organized, and user-friendly catering service platform.

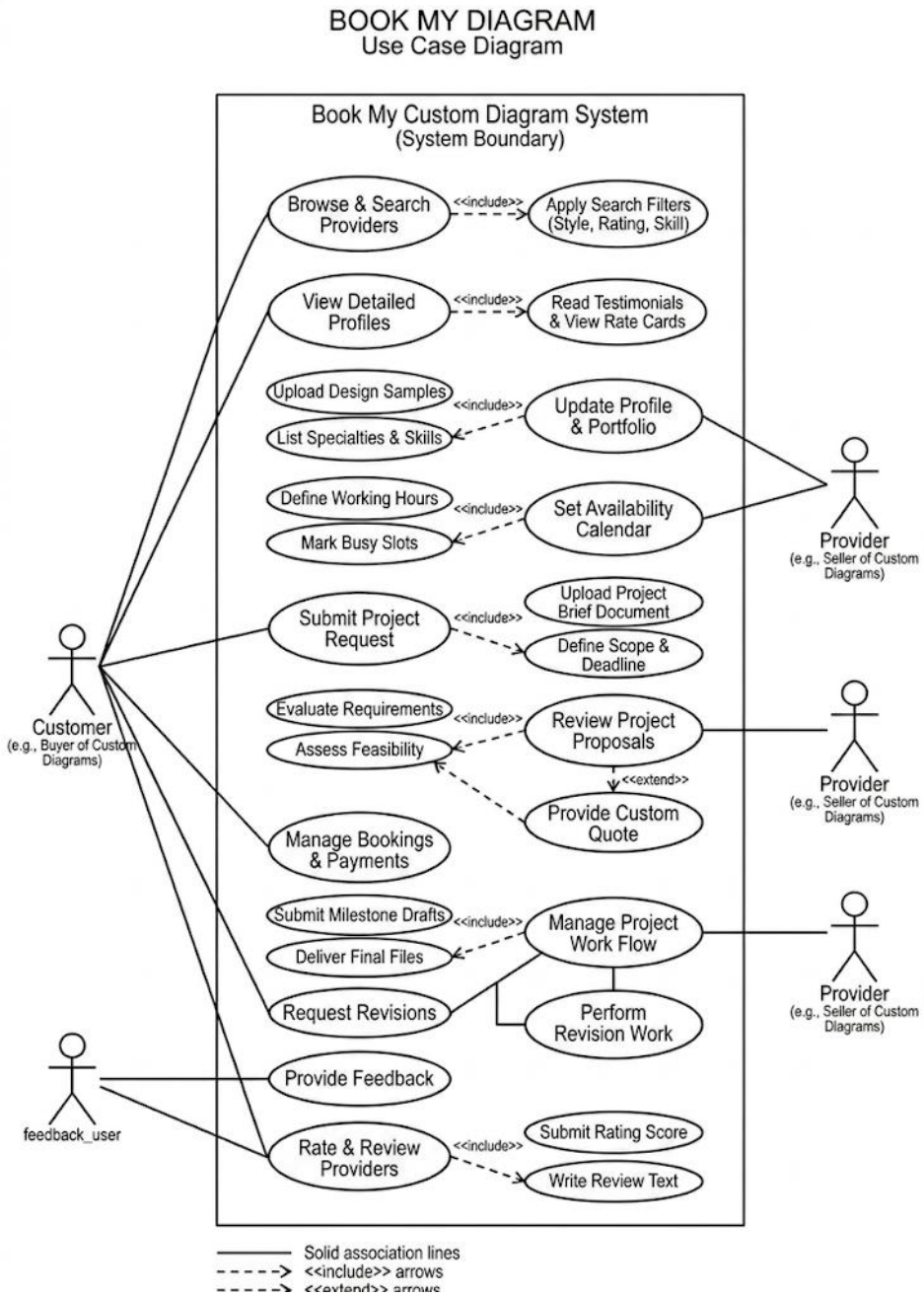


Fig 4.4.1 Use case Diagram

4.5 SEQUENCE DIAGRAM

An object's interactions are arranged chronologically in a sequence diagram. It shows the classes and objects that are a part of the scenario as well as the messages that are passed between the objects in order for the scenario to work. Sequence diagrams are commonly linked to the realizations of use cases in the Logical View of the system that is being developed. Event diagrams or event scenarios are other names for sequence diagrams.

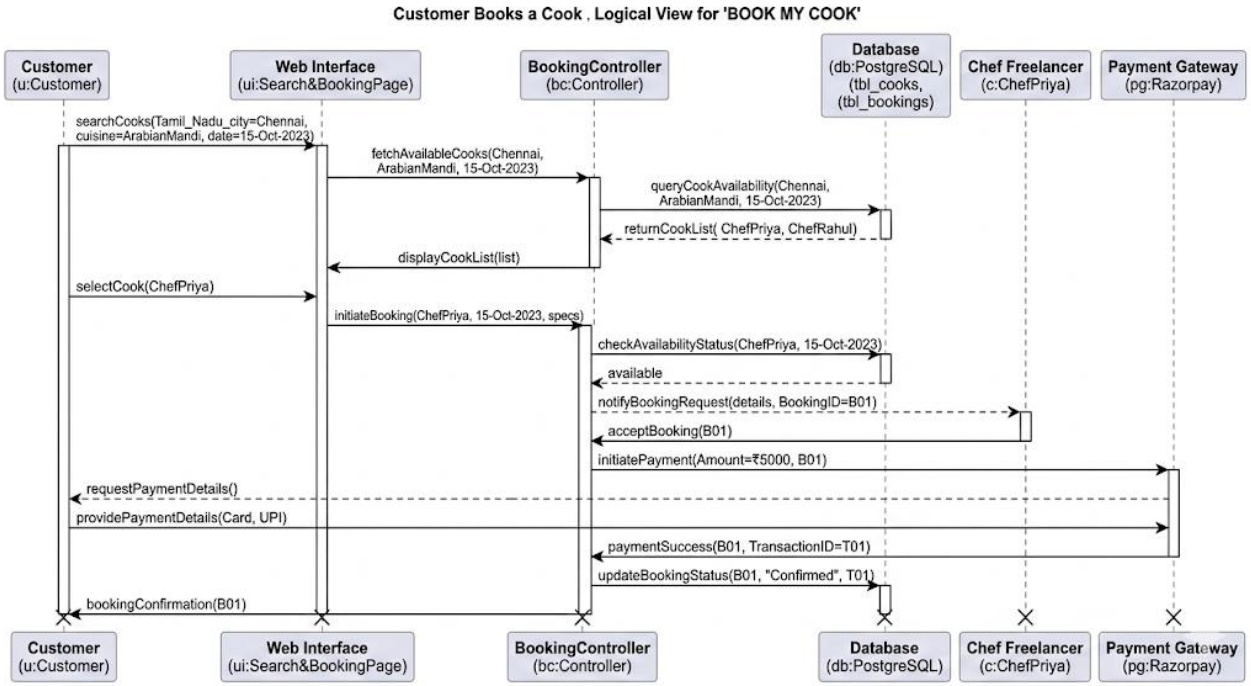


Fig 4.5.1 Sequence Diagram

4.6 CLASS DIAGRAM

A class diagram, as defined by the Unified Modeling Language (UML), is a fundamental static structural diagram that serves as the backbone of object-oriented modeling. It illustrates a system's structure by detailing its constituent classes, their properties (attributes), operations (functions), and the specific relationships and interactions between these objects. This diagram is highly versatile, serving simultaneously as a conceptual model to map out the application systematically and as a technical blueprint that directly translates into the computer code defining your system's architecture.

For your **Book My Cook** platform, the class diagram maps out the core entities required to seamlessly connect users with culinary professionals. It visually defines distinct classes such as the **Customer** (housing properties like profile data and methods like requesting a booking) and the **Cook** (detailing availability schedules, cuisine specialties, and methods like accepting or rejecting requests), while illustrating the dynamic interactions managed between them. By solidifying these data structures and their relational associations, the class diagram ensures your backend architecture and database schemas are properly organized before you begin writing the actual code.

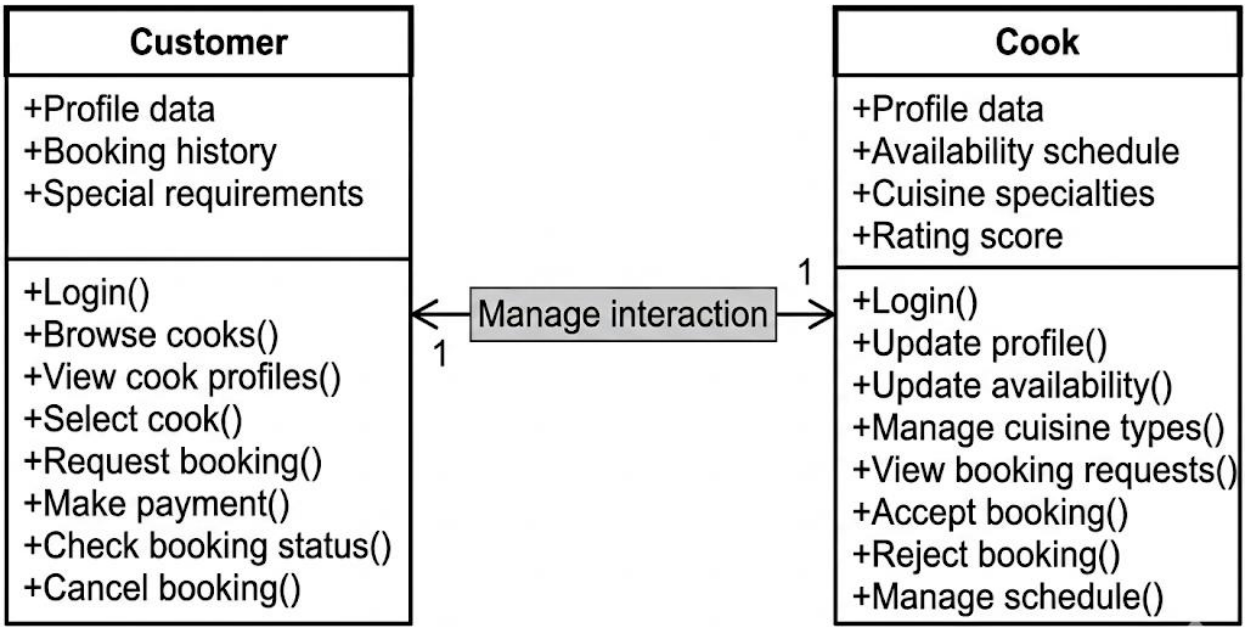


Fig 4.6.1 Class Diagram

4.7 ACTIVITY DIAGRAM

An activity diagram serves as a specialized behavioral model within UML, functioning as a sophisticated variation of a state machine that emphasizes the flow of logic across the entire system. The foundational components consist of action states, which describe atomic pieces of work or system operations, and the transitions that connect them. Unlike traditional state diagrams where external events trigger changes, transitions in this model are primarily brought about by the automatic completion or fulfillment of the action within the source state. These flows can move sequentially, through forked paths that split a single path into parallel ones, or concurrently where multiple processes occur at once.

For your **Book My Cook** project—a centralized platform designed to automate catering and event management bookings—this diagram is essential for visualizing the "Kitchen Service Process". It maps out the sequential steps a customer takes, such as logging in, browsing available cooks, and selecting cuisine preferences, while also handling parallel actions like a provider procuring ingredients and gathering equipment simultaneously. By accounting for every logical branch, including conditional paths for verifying payments or showing rejection notifications, the diagram ensures that complex interactions are fully planned before you dive into the technical implementation of your FastAPI backend and React frontend.

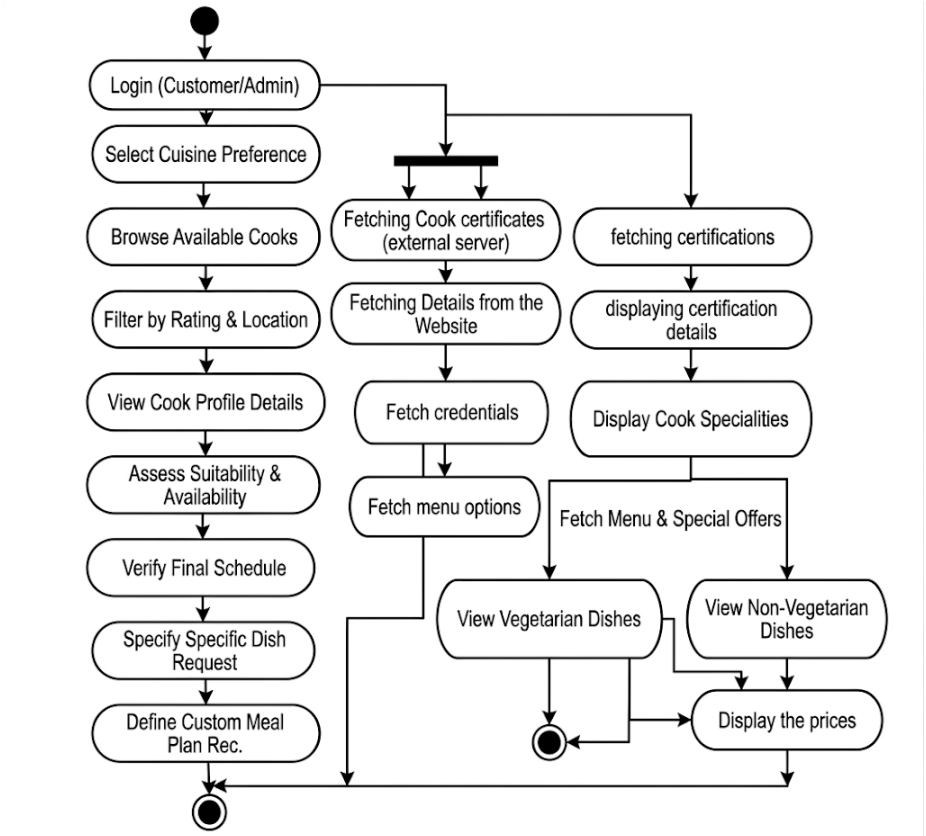


Fig 4.7.1 Activity Diagram

4.8 COLLABORATION DIAGRAM

A collaboration diagram, also known as a communication diagram in UML, focuses on the structural organization of objects that send and receive messages within a system. Unlike sequence diagrams that emphasize the chronological timeline of events, collaboration diagrams emphasize the network of relationships and direct connections between the objects themselves. By using numbered messages along these connecting lines to indicate the order of execution, this diagram provides a clear view of "who is talking to whom," making it ideal for understanding the structural dependencies required to execute a specific system operation.

For your **Book My Cook** project, a collaboration diagram effectively visualizes the network of entities involved in complex processes, such as finalizing a catering request. It illustrates how a central element, like your main Booking System, acts as an operational hub that receives input from the Customer and subsequently sends coordinated messages out to the Service Provider (Cook) for scheduling confirmation and the Payment Gateway for transaction processing. By explicitly mapping out these communication pathways, the diagram ensures your software's backend components are properly linked to handle real-world interactions seamlessly.

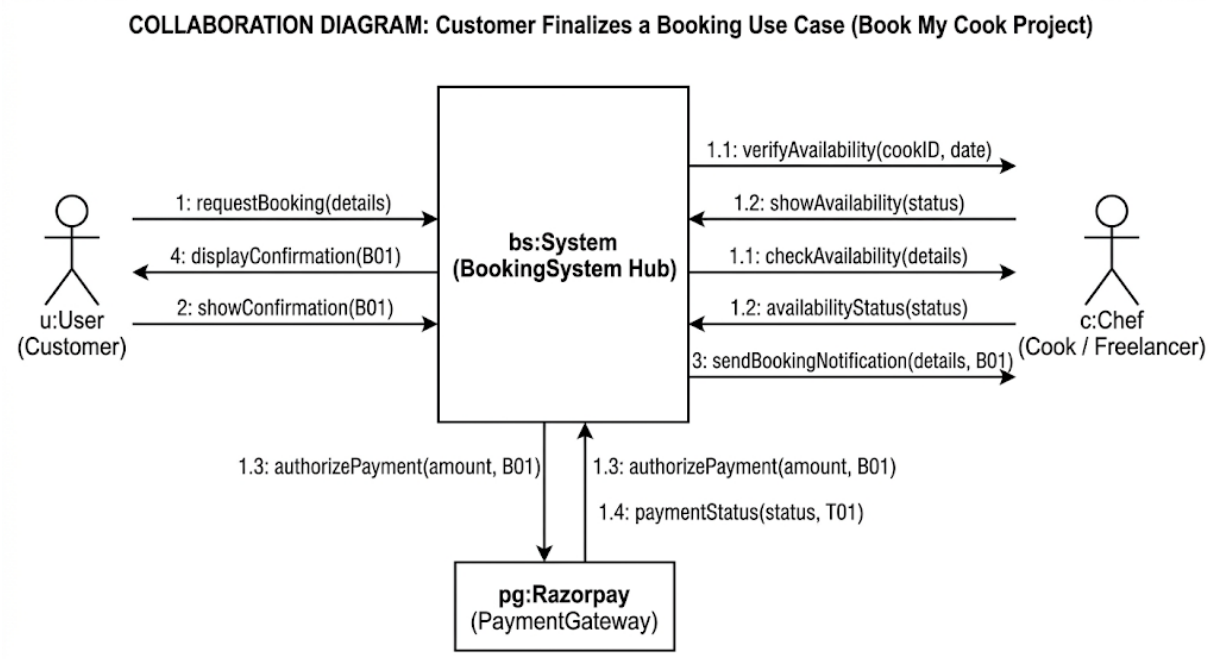


Fig 4.8.1 Collaborative Diagram

4.9 DATAFLOW DIAGRAM

A Data Flow Diagram (DFD) is a two-dimensional graphical depiction that illustrates how information is processed and transferred within a system by identifying each data source and its interactions to reach a common output. It serves as a vital visualization tool for design and development teams, requiring the identification of external inputs and outputs to determine their relationships and the resulting data transformations. By mapping these connections, the DFD helps teams analyze the system's architecture to identify inefficiencies or areas for improvement during the business development phase.

For your **Book My Cook** project—a centralized platform focused on automating catering and event management bookings—the DFD traces the journey of information as it moves between the user interface and the system's core. It visualizes how data from a **Customer's** booking request or a **Provider's** profile update flows through your **React.js** frontend into the **FastAPI** backend, where it is processed before being committed to your **PostgreSQL** database. This structural overview ensures that critical data, such as scheduling constraints and service details, is handled logically and reaches its intended destination without operational bottlenecks.

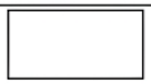
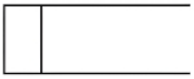
Symbol	Description
	An **entity**. A source of data or a destination for data. (e.g., A **User** interacting with the system). *This is* similar in structure to <IMAGE> but uses integrated specificity.
	A **process** or task that is performed by the system. (e.g., **'Process a Booking Request'**).
	A **data store**, a place where data is held between processes. (e.g., The **'Project Bookings'** database).
	A **data flow**. (e.g., The path of **Booking Data** flowing between a Process and a Data Store).

Table 4.9.1 Data Flow Diagram Table

LEVEL 0

Context Diagram establishes the system boundary for the entire '1.0 Book My Cook System,' showing all key external entities (Customer, Cook, Payment Gateway, and Administrator) and their high-level data flows, such as Booking Requests, Payment Auth, and System Reports.

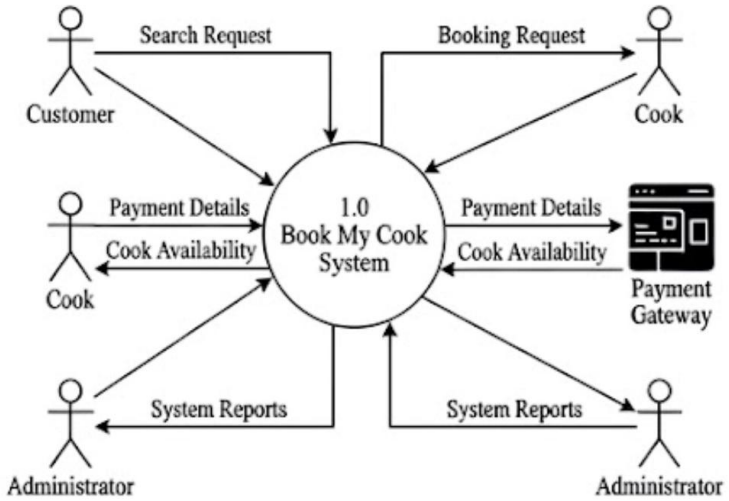


Fig 4.9.1 Level 0 DFD

LEVEL 1

Top-Level Process Decomposition breaks Process 1.0 down into distinct operational areas like 'Manage User Accounts,' 'Manage Cook Profiles,' 'Process Bookings,' and 'Process Payments,' reintroducing the specific data stores (e.g., Customer DB, Cook DB, Payment DB) where critical information is held between steps.

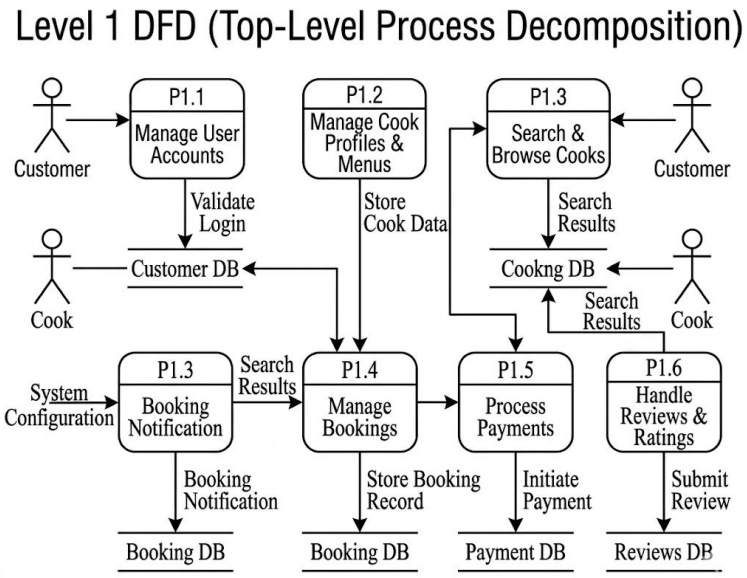


Fig 4.9.2 Level 1 DFD

LEVEL 2

Decomposition of Process 1.4 Manage Bookings dives deeper into one of the most complex Level 1 processes—'Process Bookings'—and decomposes it into specific sub-steps, including 'Validate Request,' 'Check Availability,' 'Confirm Details,' and 'Notify Stakeholders.' You can see the explicit interaction with the 'Cook DB' for schedule lookups and the 'Booking DB' for recording confirmed transactions.

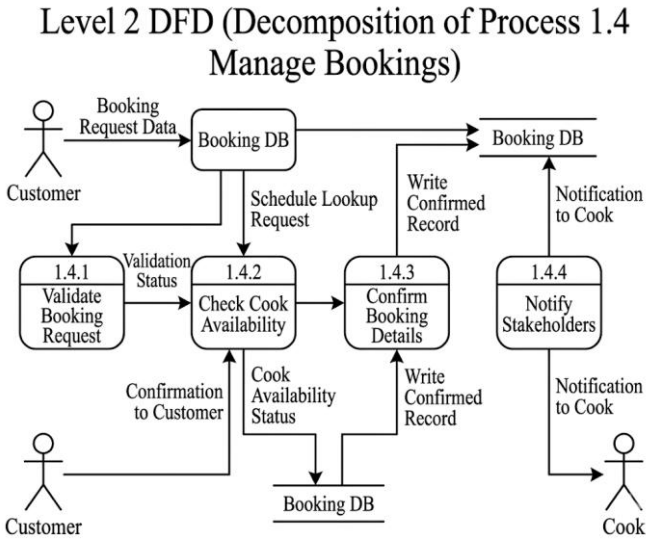


Fig 4.9.3 Level 2 DFD

LEVEL 3

Decomposition of Process 1.4.2 Check Cook Availability provides the ultimate granular detail, decomposing the 'Check Availability' sub-process from Level 2 into microscopic data operations, showing how a request is parsed, queried against the availability store, and evaluated to generate a specific status code.

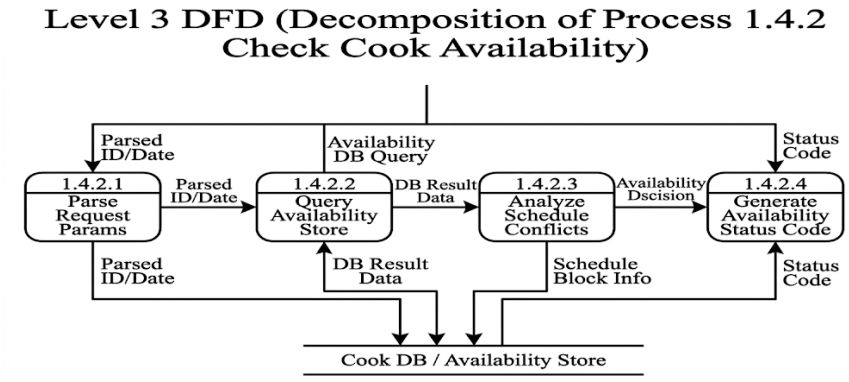


Fig 4.9.4 Level 3 DFD

CHAPTER – 5
SYSTEM SPECIFICATION

5.1 SOFTWARE REQUIREMENTS

The software environment defines the tools and technologies used for developing and running the system. The application is designed to run on any modern web browser and is built using a modern technology stack. The web interface is developed using React.js to ensure a dynamic user experience, and FastAPI is used as the backend framework to manage user interactions and business logic efficiently. The database is handled using PostgreSQL, which is used to store user accounts, cook profiles, booking details, and transaction logs. Visual Studio Code is used during development for coding and debugging. The system also integrates third-party APIs such as Razorpay for secure payment gateways and generative AI tools for platform asset creation.

5.1.1 SOFTWARE REQUIREMENTS

- Operating system : Windows OS
- Front End : React JS
- Back End : UV Python flask
- Database : PostgreSQL SERVER
- IDLE : VS Code IDLE

5.2 HARDWARE REQUIREMENTS

To run the system efficiently, certain minimum hardware specifications must be met. The application is designed to work on a personal computer or laptop with an Intel Core processor running at 2.6 GHz or higher. The server system should have at least 4 GB of RAM, although 8 GB is preferred for handling concurrent booking requests and database queries smoothly. A minimum of 320 GB of hard disk space is required to store application components, user portfolios, and extensive database logs. A standard color monitor ensures proper visibility of the user dashboard, schedules, and interfaces. Basic peripherals such as a standard keyboard and mouse are needed for user interaction. Internet connectivity is essential for accessing the web platform and processing live payments. For accessing the service, a standard mobile phone or PC can be used by both customers and cooks.

5.2.1 HARDWARE REQUIREMENTS

- Processor : Intel core processor 2.6.0 GHZ
- RAM : 4 GB
- Hard disk : 320 GB
- Compact Disk : 650 Mb

CHAPTER – 6
SOFTWARE DESCRIPTION

6.1 INTRODUCTION

The proposed catering and event service booking system, "Book My Cook," is a web-based application developed using Python FastAPI and React.js for seamless culinary service discovery and marketplace management. The system connects customers with verified service providers and analyzes user preferences using Natural Language Processing and heuristic scoring techniques. The AI-powered chatbot, "Cheffy," identifies user intent, location, and budget constraints from text inputs. Rule-based heuristic algorithms are used to filter, rank, and recommend service providers based on proximity, ratings, and availability. React.js and Tailwind CSS are used to design an interactive and responsive Single-Page Application (SPA) dashboard for customers, cooks, and administrators. PostgreSQL is used to store user portfolios, booking records, schedules, and transaction details. The system also provides real-time notifications and bidirectional messaging using WebSockets and integrates Razorpay for secure financial transactions to support efficient booking operations.

6.2 FRONTEND: React.js, Tailwind CSS

The frontend of the proposed "Book My Cook" platform is designed and developed using React.js and Tailwind CSS technologies. The frontend acts as the communication layer between the users and the backend API modules. It provides an interactive and user-friendly environment where customers, service providers (chefs, caterers, decorators), and administrators can manage bookings and portfolios in real time. The frontend interface is designed to ensure simplicity, responsiveness, and efficient navigation across different devices and screen sizes.

React.js is used to build a dynamic Single-Page Application (SPA) through its component-based architecture. Different React components are developed for modules such as multi-role authentication, service discovery, provider portfolios, booking calendars, and the AI chatbot interface. React Router manages client-side navigation seamlessly without page reloads. The service discovery dashboard displays available cooks, categorized by cuisine, event type, and location, complete with high-quality portfolio images and verified ratings.

Tailwind CSS is used as the utility-first framework to enhance the visual appearance of the application. It improves the overall user interface by applying consistent colors, spacing, typography, and responsive grid layouts directly within the React components. The styling helps users easily navigate the platform, from browsing menus to completing payments. Tailwind CSS

ensures responsive layouts so that the application adapts automatically to different screen resolutions, improving accessibility for users on mobile devices, tablets, or laptops.

The frontend dashboard contains several critical modules. The multi-role login module provides distinct, secure authentication paths for customers, providers, and admins. The service management module allows cooks to build their portfolios, set availability calendars, and define pricing. The booking interface enables customers to select time slots and submit requests smoothly. The real-time messaging panel facilitates direct communication between customers and providers. Notification components and interactive chat windows for the "Cheffy" AI assistant are also integrated directly into the user interface.

The frontend is connected with the FastAPI backend using RESTful API calls and WebSocket connections. React dynamically fetches and renders processed data from the backend endpoints, allowing real-time updates of booking statuses and incoming messages without manual page reloads. State management tools handle data flow within the application, ensuring smooth communication between the AI recommendation engine, the database, and the user interface.

The frontend implementation also focuses on usability and accessibility. Clear navigation menus, interactive calendar pickers, readable fonts, and organized dashboards help users access service information quickly. Alert components improve the visibility of important notifications, such as booking confirmations or payment requests. Overall, the frontend implementation plays a major role in presenting marketplace information clearly and interactively, acting as the visual control center for the "Book My Cook" platform.

6.3 BACKEND: Python, Flask

The backend of the "Book My Cook" platform is implemented using the Python programming language and the Flask web framework. The backend is responsible for handling user authentication, business logic, AI chatbot processing, database management, booking conflict resolution, and payment gateway integration. It acts as the core processing unit of the entire system and integrates all major technologies used in the project.

Python is selected as the primary programming language because of its simplicity, flexibility, and extensive support for artificial intelligence and data processing libraries. Flask is utilized as the backend web framework for building robust RESTful APIs. Flask provides a lightweight and flexible architecture, handling HTTP requests, routing, and responses efficiently. The framework manages secure URL routing, session handling (via JSON Web Tokens or Flask-Login), form submissions, and seamless communication between the React frontend and the backend services.

Natural Language Processing (NLP) techniques are integrated into the backend to power the "Cheffy" AI chatbot. Python NLP libraries (such as NLTK or spaCy) are used to parse user queries, detect intent (e.g., "find a caterer," "check booking status"), and extract specific entities such as location, budget, and cuisine preferences. These NLP pipelines process customer text inputs to provide automated, context-aware assistance, reducing the need for manual customer support and guiding users through the booking process efficiently.

A rule-based heuristic recommendation engine is implemented within the backend logic. This engine analyzes customer search parameters and matches them against the service provider database. The algorithm calculates matching scores based on factors like geographic proximity (location-based filtering), average review ratings, calendar availability, and pricing constraints. This ensures that users receive highly relevant and personalized culinary service suggestions in real time.

PostgreSQL and SQLAlchemy ORM

PostgreSQL is used as the robust, highly scalable relational database management system for the platform. The database securely stores user profiles, hierarchical location data, service portfolios, booking states, chat histories, and payment logs.

To bridge the Python backend with the PostgreSQL database, **SQLAlchemy** is utilized as the Object-Relational Mapper (ORM). SQLAlchemy ORM allows developers to interact with the database using Python classes and objects rather than writing raw SQL queries. It handles complex database operations, table relationships (such as linking a customer entity to a booking and a cook entity), and automated schema migrations. By using SQLAlchemy ORM, the backend ensures data consistency, prevents SQL injection attacks, and provides an efficient layer for executing advanced queries required by the recommendation engine and booking conflict resolution algorithms.

The backend implementation includes comprehensive error handling, input validation, and enterprise-grade security mechanisms. Role-Based Access Control (RBAC) ensures that users can only access endpoints appropriate for their assigned role (Customer, Cook, or Admin). Secure password hashing (using libraries like Werkzeug or Bcrypt) and encryption protocols protect sensitive user data. Logging mechanisms monitor system activities to maintain operational records for auditing and debugging purposes.

Overall, the backend implementation forms the operational and intelligent core of the "Book My Cook" platform. It seamlessly integrates flexible Flask API routing, AI-driven chatbot processing, robust PostgreSQL database management via SQLAlchemy ORM, secure payment gateways, and real-time communication protocols into a unified digital marketplace.

CHAPTER – 7
IMPLEMENTATION

7.1 IMPLEMENTATION

The implementation of "Book My Cook" translates the proposed system design into a fully functional, AI-powered digital marketplace for catering and event services. This chapter provides a comprehensive account of the complete implementation workflow—from environment setup and database configuration, through backend API development, frontend component integration, AI chatbot logic implementation, and final system deployment. The implementation follows a structured, modular approach where each component (User Management, Service Discovery, Booking Engine, Payment Integration, and AI Assistance) is independently developed, tested, and integrated into the unified "Book My Cook" platform.

The system is implemented using a modern technology stack. Python serves as the primary programming language for the backend, utilizing the high-performance FastAPI framework for routing and business logic. Natural Language Processing (NLP) libraries (such as NLTK or spaCy) power the intent recognition and entity extraction for the "Cheffy" AI chatbot. The frontend is built as a Single-Page Application (SPA) using React.js and styled with Tailwind CSS for responsive design. PostgreSQL manages all persistent data storage, accessed via the SQLAlchemy Object-Relational Mapper (ORM). Pydantic handles data validation and serialization. Security is maintained through JWT (JSON Web Tokens) for session management and PassLib/Bcrypt for password hashing. Real-time features are implemented using WebSockets. Together, these technologies form a cohesive, fully integrated platform capable of providing a seamless, real-time culinary service booking experience.

7.1.2 Implementation Workflow

The complete "Book My Cook" implementation follows a seven-stage workflow as described below:

Stage 1 — Environment Setup and Database Schema Configuration

Stage 2 — Backend Authentication and User Management Implementation

Stage 3 — Service Portfolio and Recommendation Engine Development

Stage 4 — Core Booking Engine and WebSocket Integration

Stage 5 — AI Chatbot ("Cheffy") Logic Implementation

Stage 6 — Frontend SPA Development and State Management

Stage 7 — Payment Gateway Integration and Testing

7.1.3 Environment Setup and Database Schema Configuration

The implementation process begins with establishing the development environment. A Python virtual environment is created, and all necessary dependencies (FastAPI, Uvicorn, SQLAlchemy, Pydantic, PassLib) are installed via pip.

The foundational step is the design and implementation of the PostgreSQL database schema. Using SQLAlchemy ORM, database models (tables) are defined as Python classes. The primary tables implemented include:

Users: Stores credentials, roles (Customer, Cook, Admin), and contact details.

Portfolios: Linked to Cook users, storing service types, cuisine specialties, base pricing, and media URLs.

Bookings: Stores event dates, guest counts, assigned Cook IDs, Customer IDs, and booking statuses (Pending, Confirmed, Paid).

Reviews: Stores ratings and textual feedback linked to completed bookings.

Messages: Stores chat histories for WebSocket communication. Alembic is utilized to manage database migrations, ensuring that the defined SQLAlchemy models accurately reflect the physical PostgreSQL tables.

7.1.4 Backend Authentication and User Management Implementation

Security is implemented first to protect subsequent API endpoints. The user registration flow is developed, capturing role-specific data. PassLib with Bcrypt is used to hash user passwords before storing them in the database, ensuring raw passwords are never saved.

The login mechanism is implemented using OAuth2 with Password (and hashing). Upon successful credential verification, FastAPI generates a JSON Web Token (JWT) containing the user's ID and role. This token is returned to the frontend and must be included in the Authorization header of all subsequent API requests. Role-Based Access Control (RBAC) dependency functions are created in FastAPI (e.g., get_current_admin, get_current_cook) to restrict access to specific endpoints based on the token's payload.

7.2.3 Service Portfolio and Recommendation Engine Development

RESTful CRUD (Create, Read, Update, Delete) endpoints are implemented for service providers to manage their portfolios. This includes endpoints for uploading profile images (handling file storage locally or via a cloud bucket) and updating availability calendars.

For the customer side, the recommendation engine is implemented. Instead of complex deep learning models, a highly optimized, rule-based heuristic search algorithm is written in Python. This algorithm takes search parameters (location, event type, budget) from the frontend via query parameters. It uses SQLAlchemy to execute complex joins across the Portfolios, Users, and Reviews tables to filter out unavailable or mismatched cooks. The algorithm then ranks the results based on average ratings and proximity, returning a serialized JSON list of recommended service providers to the client.

7.1.5 Core Booking Engine and WebSocket Integration

he core transactional logic is implemented in the booking module. Endpoints are created for customers to submit booking requests. The backend performs validation checks against the requested Cook's availability calendar to prevent double-booking conflicts.

Simultaneously, real-time bidirectional communication is established. FastAPI's WebSocket support is implemented to create a persistent connection between a customer and a cook. When a user connects to the /ws/{booking_id} endpoint, their connection is added to a connection manager. Messages sent through the socket are broadcast to the recipient in real-time and concurrently saved to the Messages PostgreSQL table to maintain conversation history.

7.1.6 AI Chatbot ("Cheffy")

Logic Implementation The "Cheffy" AI module is developed as a separate backend service layer. A dedicated FastAPI endpoint receives user text inputs from the frontend chat interface.

Intent Recognition: Python NLP libraries (such as spaCy) process the text string to classify the user's intent (e.g., classifying "I need a chef for a birthday" as a search_service intent).

Entity Extraction: The NLP pipeline extracts specific parameters (e.g., extracting "birthday" as event_type).

Response Generation: Based on the identified intent, the backend triggers specific internal functions. If the intent is a search, it automatically calls the Recommendation Engine (Stage 3) and formats the top results into a conversational text response, which is sent back to the user interface.

7.1.7 Frontend SPA Development and State Management

With the backend APIs established, the React.js frontend is developed. The application is structured into modular components (Navbar, ServiceCard, BookingModal, ChatWindow). React Router is implemented to handle navigation between the Customer Dashboard, Cook Dashboard, and Admin Control Panel without page reloads. State management (utilizing React Context API or Redux) is configured to manage the user's authentication status globally across the

application. Axios is integrated to handle HTTP requests to the FastAPI backend, automatically appending the JWT token to secure requests. Tailwind CSS is applied at the component level to rapidly style the user interface, ensuring mobile responsiveness and a modern aesthetic.

7.1.8 Payment Gateway Integration and Deployment

The final implementation stage involves financial processing. The Razorpay Python SDK is integrated into the backend. When a booking is confirmed, the backend generates a Razorpay Order ID. This ID is passed to the React frontend, which triggers the Razorpay checkout modal. Upon successful payment by the customer, Razorpay sends an encrypted webhook payload back to a dedicated FastAPI endpoint. The backend verifies the webhook signature for security, updates the booking status to 'Paid' in the PostgreSQL database, and triggers an automated confirmation notification to both the customer and the cook.

7.2 ALGORTHM

7.2.1 RULE-BASED RECOMMENDATION ENGINE

A rule-based recommendation engine is a subset of information retrieval systems that provides personalized suggestions based on predefined rules and heuristic algorithms. It is one of the various types of recommendation systems used for different applications and data types. A rule-based engine uses keyword matching, intent detection, and weighted scoring algorithms to generate relevant recommendations without requiring machine learning model training. There are other types of recommendation systems in modern applications, but for service marketplace platforms where interpretability and real-time performance are critical, rule-based engines are the architecture of choice. This makes them highly suitable for service discovery tasks and for applications where transparency and explainability are vital, such as booking platforms and e-commerce systems.

7.2.2 HEURISTIC SCORING ALGORITHM

A heuristic scoring algorithm is a problem-solving approach that uses practical methods and weighted criteria to evaluate and rank items based on multiple factors. The algorithm assigns numerical scores to each item by calculating weighted contributions from various attributes such as ratings, review counts, location proximity, price compatibility, verification status, and availability. The scoring formula typically follows a linear combination model where each factor is multiplied by its assigned weight and summed to produce a final score. Items are then sorted in descending order by score to identify top recommendations. This approach is computationally efficient, easily interpretable, and allows for fine-tuning of factor weights to optimize recommendation quality for specific use cases.

PROCEDURE

Step 1 : Initialize the scoring weights for each factor (rating: 40%, reviews: 20%, location: 10%, price: 10%, verification: 10%, status: 10%).

Step 2 : Set the maximum score limit for each factor.

Step 3 : For each service in the database do:

Step 4 : Calculate rating score as (service.rating × 8) with maximum 40 points.

Step 5 : Calculate review score as (service.review_count / 5) with maximum 20 points.

Step 6 : Add 10 points if service city matches user-specified location.

Step 7 : Add 10 points if service price is within or below user budget.

Step 8 : Add 10 points if service is verified by administrator.

Step 9 ; Add 10 points if service is currently active.

Step 10 : Sum all factor scores to compute total recommendation score.

Step 11 : Sort all services by total score in descending order.

Step 12 : Return top N services as recommendations to the user.

Step 13 : Handle edge cases where no services match the filtering criteria.

Step 14 : Format and display recommendations with relevant service details.

7.2.3 INTENT DETECTION USING KEYWORD MATCHING

Intent detection using keyword matching is a natural language understanding technique that identifies user intentions by analyzing the presence of predefined keywords in user messages. The system maintains a dictionary of intent categories, each associated with a set of representative keywords and phrases. When a user message is received, the algorithm performs case-insensitive string matching to detect which keywords appear in the message, and maps them to corresponding intents. This approach is particularly effective for domain-specific chatbots where the vocabulary is limited and well-defined. It provides fast, deterministic results without requiring complex machine learning models, making it suitable for real-time chatbot applications where response latency is critical.

7.2.4 MULTI-VERSION CONCURRENCY CONTROL (MVCC)

Multi-Version Concurrency Control (MVCC) is a database optimization technique used by PostgreSQL to handle concurrent access to data without locking conflicts. MVCC allows multiple transactions to access the same data simultaneously by maintaining multiple versions of each row. When a transaction modifies data, PostgreSQL creates a new version rather than overwriting the existing one, enabling readers to access the previous version while writers create new versions. This approach eliminates read-write conflicts and provides snapshot isolation, ensuring that readers see a consistent view of the database at a specific point in time. MVCC is essential for high-concurrency web applications where multiple users simultaneously read and write booking, payment, and message data.

7.2.5 AES-256 SYMMETRIC ENCRYPTION

AES-256 (Advanced Encryption Standard with 256-bit key) is a symmetric encryption algorithm used to protect sensitive data at rest. It operates on fixed-size blocks of data using a series of substitution, permutation, and mixing operations defined by the encryption key. The Fernet implementation in Python's cryptography library provides secure AES-256 encryption with message authentication, ensuring both confidentiality and integrity of encrypted data. The algorithm uses a 256-bit key derived from a base64-encoded secret, making it resistant to brute-force attacks. AES-256 encryption is widely adopted for field-level encryption in database systems where sensitive information such as payment details, personal identification, and private messages must be protected from unauthorized access.

7.2.6 JWT (JSON WEB TOKEN) AUTHENTICATION

JSON Web Token (JWT) is a compact, URL-safe means of representing claims between two parties for authentication and authorization purposes. A JWT consists of three parts: a header specifying the token type and signing algorithm, a payload containing user claims and metadata, and a signature ensuring token integrity. The token is generated upon successful authentication and sent to the client, which includes it in subsequent API requests. The server validates the token signature and extracts user identity without requiring database lookups, enabling stateless authentication. JWT supports configurable expiration times, refresh token mechanisms, and role-based claims, making it suitable for securing RESTful APIs in modern web applications where scalability and distributed architecture are required.

7.2.7 IMPORTANCE OF RULE-BASED RECOMMENDATION ENGINE

There are several benefits of using a rule-based recommendation engine for service discovery tasks:

Interpretability: Rule-based engines provide transparent and explainable recommendations, allowing users and developers to understand exactly why specific services are suggested. This makes them well-suited for platforms where trust and accountability are essential.

Real-time Performance: Rule-based engines generate recommendations instantly without requiring model training or complex computations, enabling immediate response to user queries and dynamic filtering based on real-time data.

No Training Data Required: Unlike machine learning models, rule-based engines do not require large datasets for training, making them easy to deploy and maintain for new platforms with limited historical data.

Domain-Specific Optimization: The scoring weights and rules can be precisely tuned for specific business requirements, allowing fine-grained control over recommendation priorities such as ratings, location, or price.

Scalability: Rule-based engines scale efficiently with growing service catalogs, as recommendations are computed on-demand without storing pre-computed models or embeddings.

Deterministic Results: The same input always produces the same output, ensuring consistent user experience and enabling reliable testing and debugging of recommendation logic.

Rule-based recommendation engines have proven effective for service marketplace platforms where transparency, speed, and domain-specific customization are critical requirements.

PROCEDURE

Step 1 : Initialize System Load intent keyword dictionaries and define scoring weights.

Step 2 : Preprocess Input Convert the incoming user message to lowercase.

Step 3 :Detect Intents Scan for keywords to identify service requests or special intents (e.g., greetings, FAQs).

Step 4 : Extract Entities Parse the message for specific locations, event types, cuisines, and numeric budgets.

Step 5 : Query Database Fetch services that match the detected intent and service type.

Step 6 :Calculate Scores Evaluate each matched service using the weighted factors:

- Rating (max 40 pts)
- Review count (max 20 pts)
- Location match (10 pts)
- Budget match (10 pts)
- Admin verification (10 pts)
- Active status (10 pts)

Step 7 : Rank Results Sort the services in descending order based on their total calculated score.

Step 8 : Return Output Deliver the top 5 recommendations, or provide an appropriate fallback response if no services match.

CHAPTER – 8

RESULT

8.1 SCREENSHOT

This is the most common term in web design and marketing. It refers to the specific public-facing page users "land" on to learn about your app and be convinced to sign up or log in.

Fig 8.1.1 Home page Before Login

Often used by UX/UI designers to describe the specific view where the user sets up their new profile for "My Book My Cook."

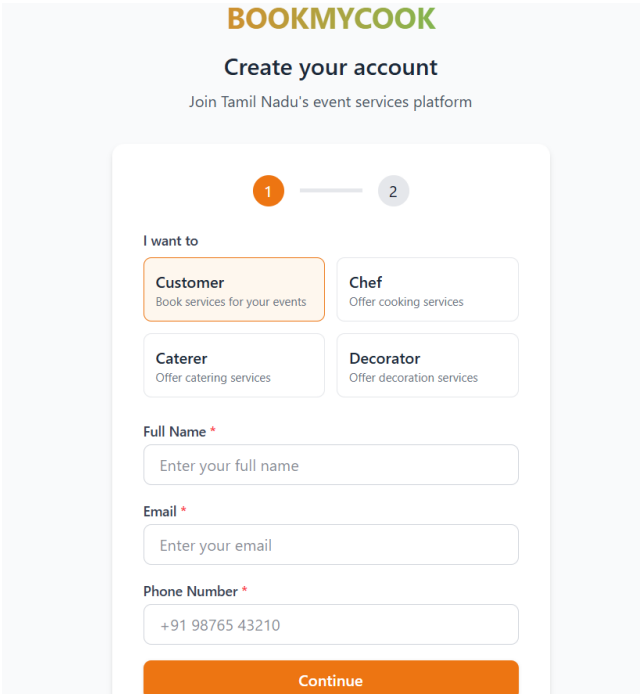


Fig 8.1.2 Sign up Page

Often used interchangeably with "Login," but many major tech companies (like Google and Apple) prefer "Sign In" as their standard UI design pattern.

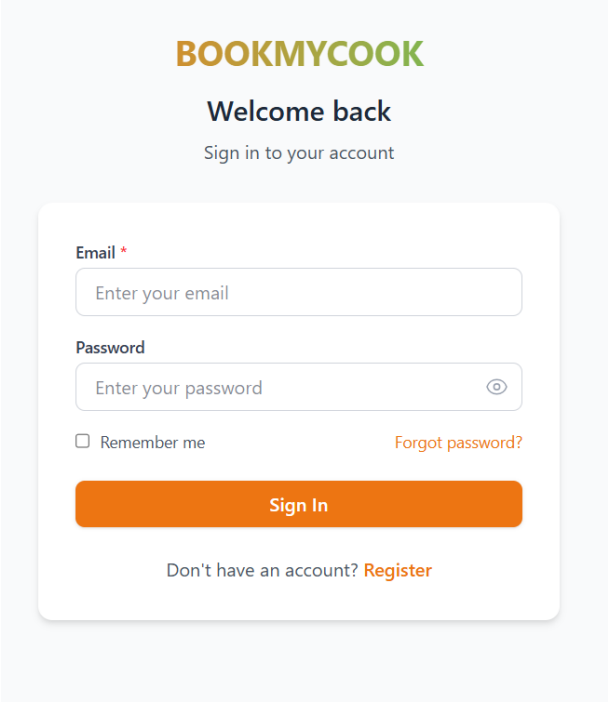


Fig 8.1.3 Login Page

Because the main content consists of a grid of cards offering different services (Chettinad Feast, Fusion Tamil Cuisine, etc.), this is the most accurate term for the *content* of the page. In e-commerce or booking platforms, pages that display a grid of products or services are called Listing Pages.

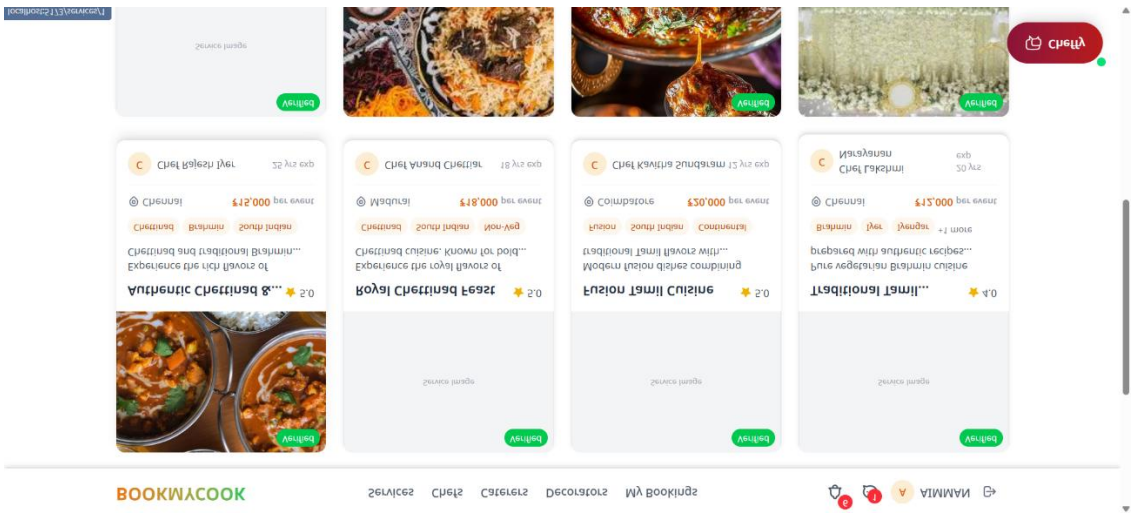


Fig 8.1.4 Listing Page

This is a very specific UX/UI industry term. "Facets" are the different categories or attributes a user can use to narrow down their search (e.g., City, Cuisine, Price, Rating). Because your users can apply multiple filters at once to refine their results, they are using faceted search.

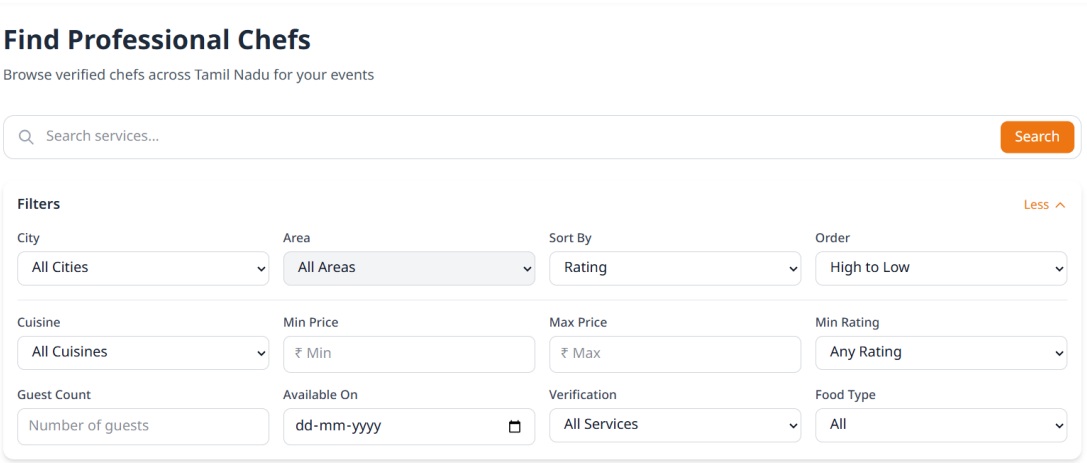


Fig 8.1.5 Filter Panel

In e-commerce and booking platforms, the page that provides the deep dive into a specific item, including images, pricing, and descriptions, is universally called a PDP (Product Details Page). Since your platform focuses on booking chefs, developers will often adapt this to SDP (Service Details Page).

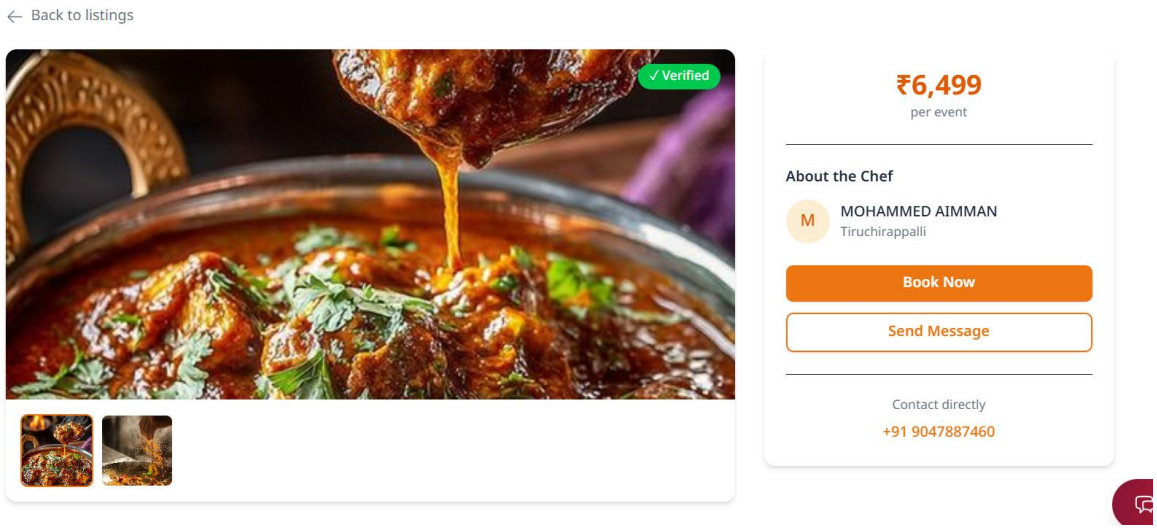


Fig 8.1.6 Service Details Page

This refers to the main text paragraph detailing the chef's experience, skills, and background.

Fig 8.1.7 Service Description Page

Booking Calendar / Availability Calendar: This is the most direct and common industry term for this specific use case, as it allows users to visually check when a service provider is free.

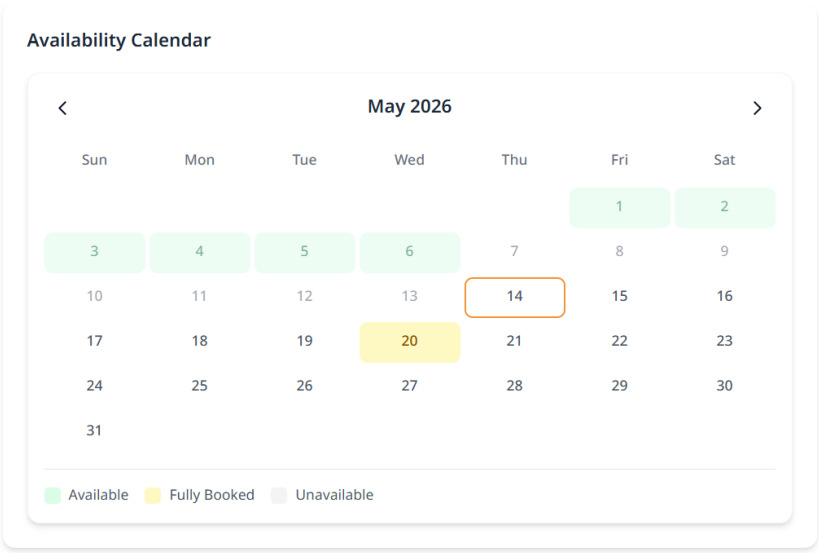


Fig 8.1.8 Booking Calendar / Availability Calendar

From a data perspective, developers would call this a "history feed" because it lists past transactions or events associated with this specific chef or service provider, arranged chronologically.

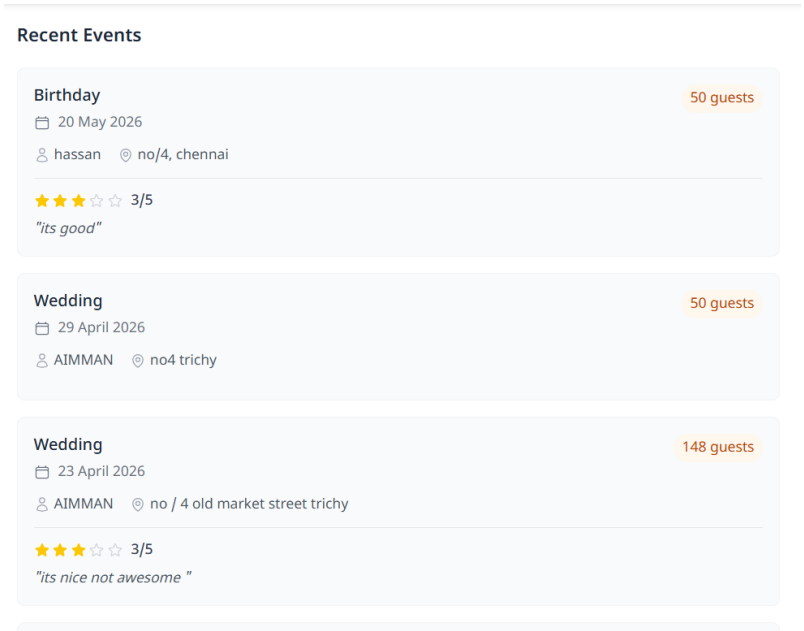


Fig 8.1.9 Event History

This is the most standard and widely understood term across both development and design. It is a dedicated block that displays user feedback.

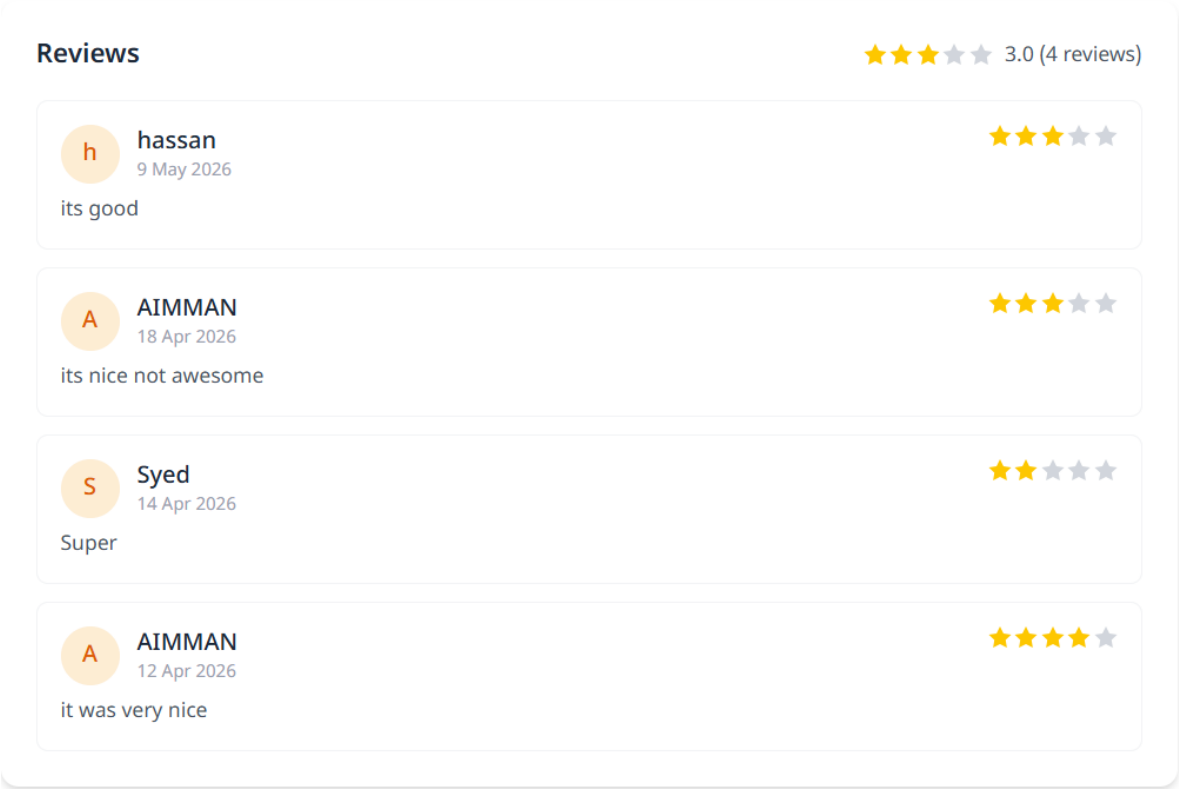


Fig 8.1.10 User Feedback

This is the most common industry term. Because it usually opens up from a small icon in the corner of the screen and acts as a self-contained window, it is referred to as a "widget" or "modal."

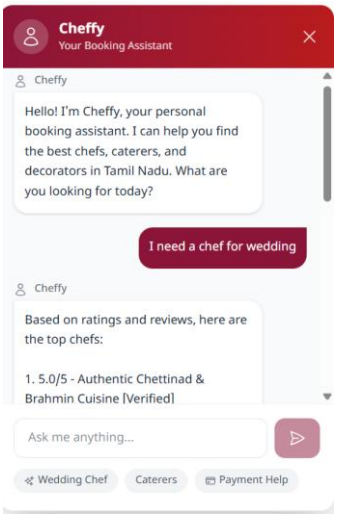


Fig 8.1.11 Cheffy chatbot

This is the Account Settings or Edit Profile page, where users manage their personal information and update their security credentials.

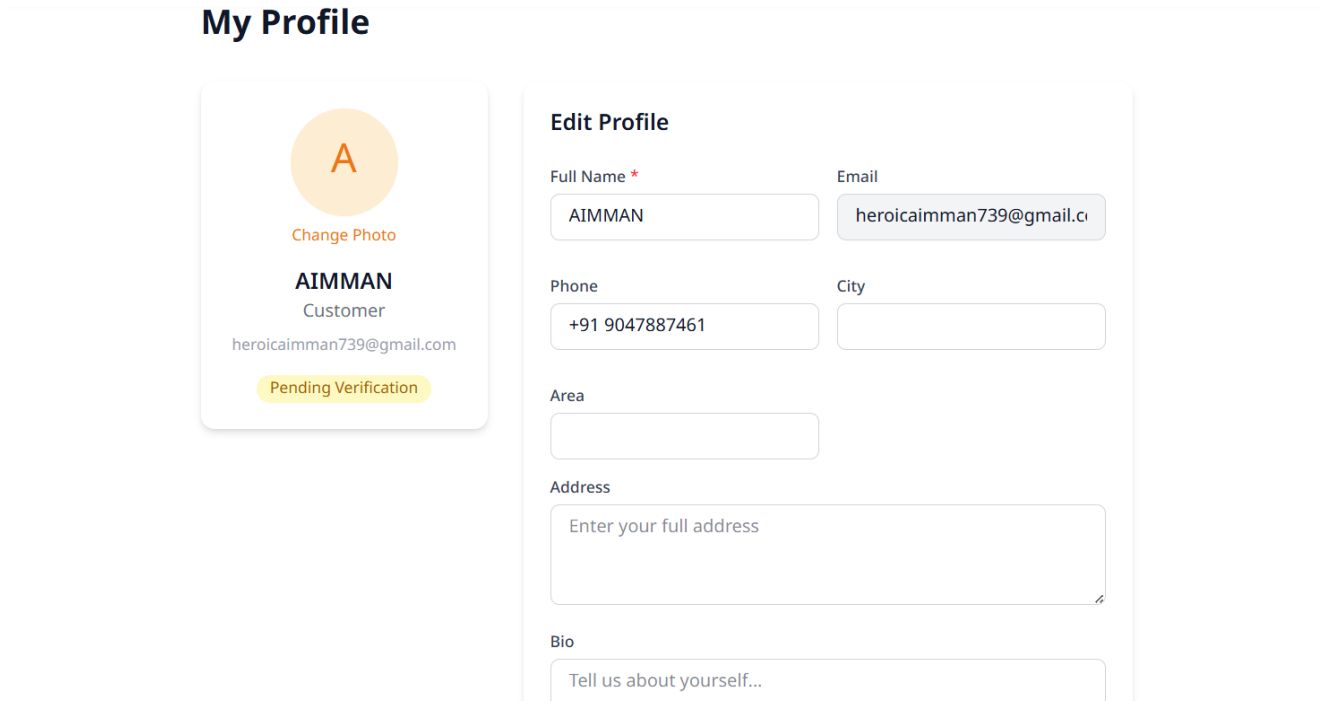


Fig 8.1.12 Edit profile Page 1

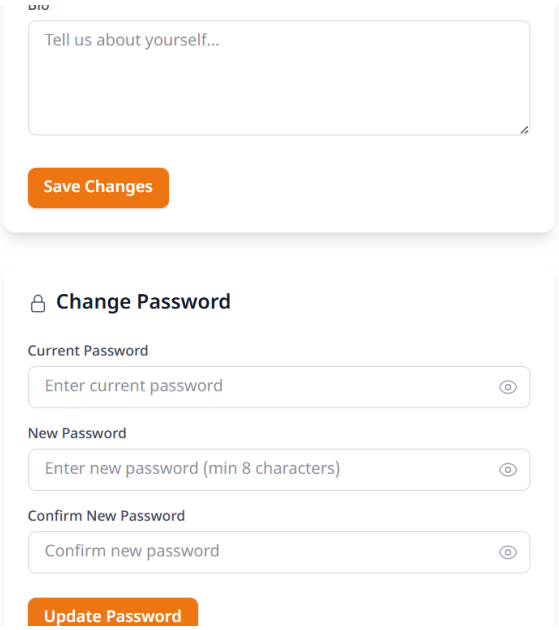


Fig 8.1.13 Edit Profile Page 2

This is the Direct Messaging (DM) Interface or Conversation View, where users engage in one-on-one chat threads with service providers.

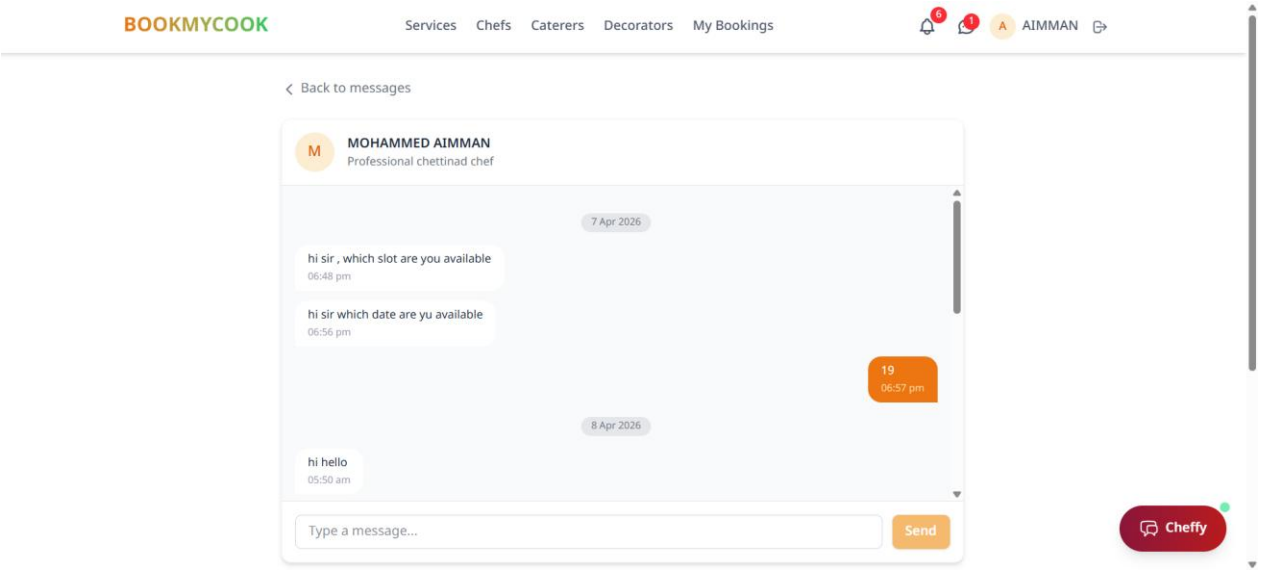


Fig 8.1.14 Direct Message Section

This is the Notifications Center or Activity Feed, where users view and manage their system alerts and booking updates.

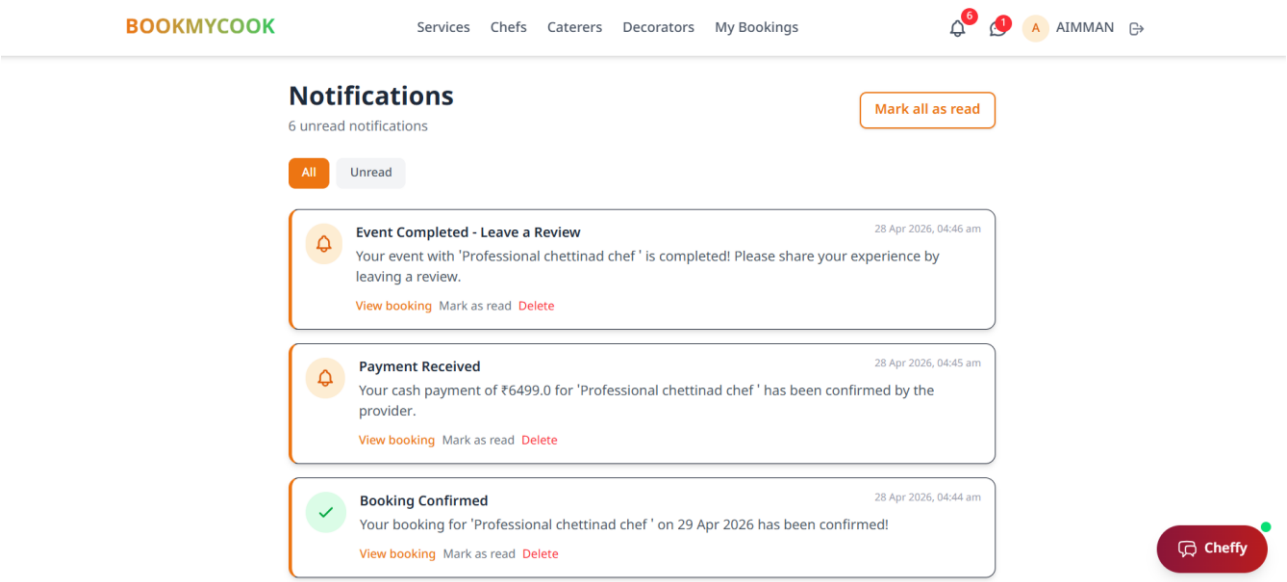


Fig 8.1.15 Notification Section

This is the Booking Details Form or Checkout Interface, where users input their specific event requirements as part of a multi-step reservation flow.

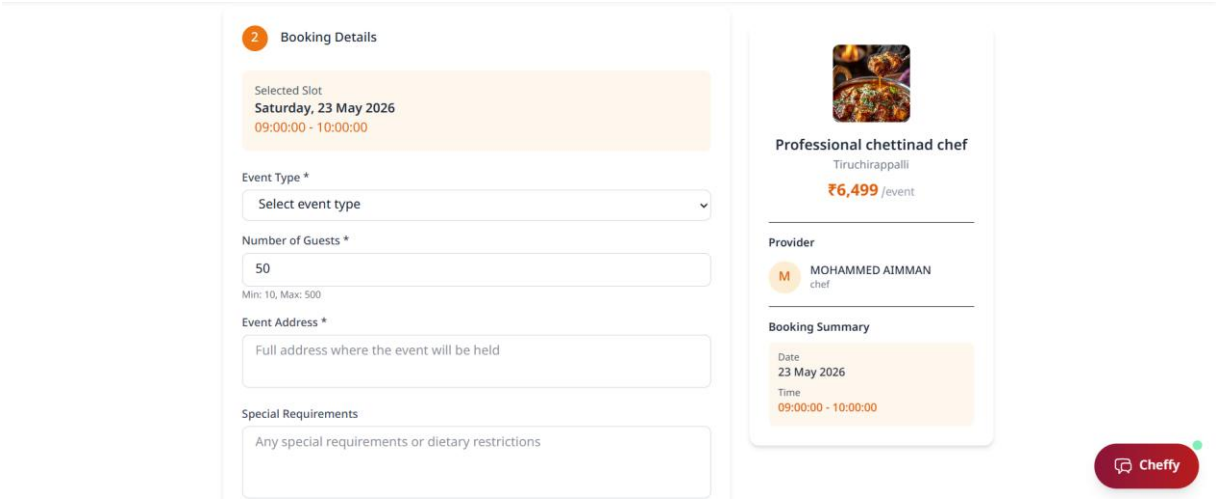


Fig 8.1.16 Booking Details Form

This is the Booking Details Page or Reservation Summary, providing a read-only overview of a specific event request and its current status.

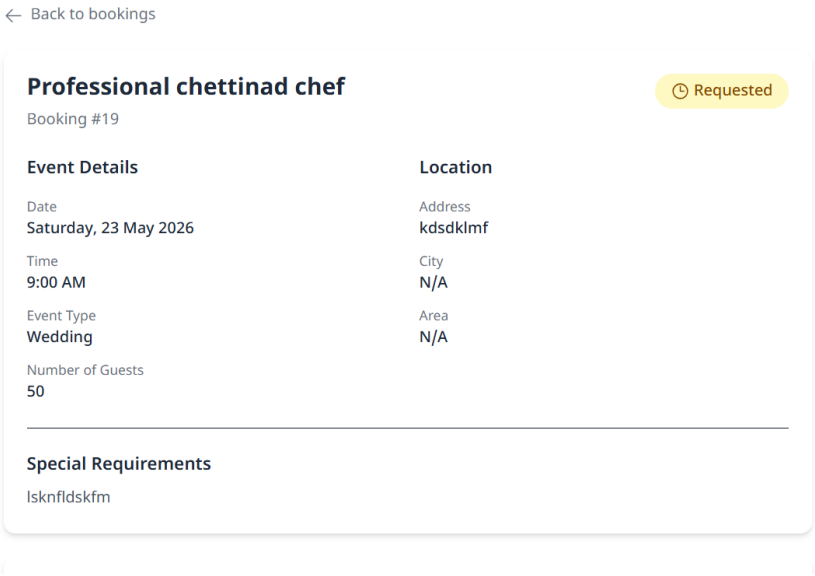


Fig 8.1.17 Booking Summary

This is the Payment Summary and Provider Contact section of the reservation view, featuring the final cost breakdown and a destructive Cancel Booking action.

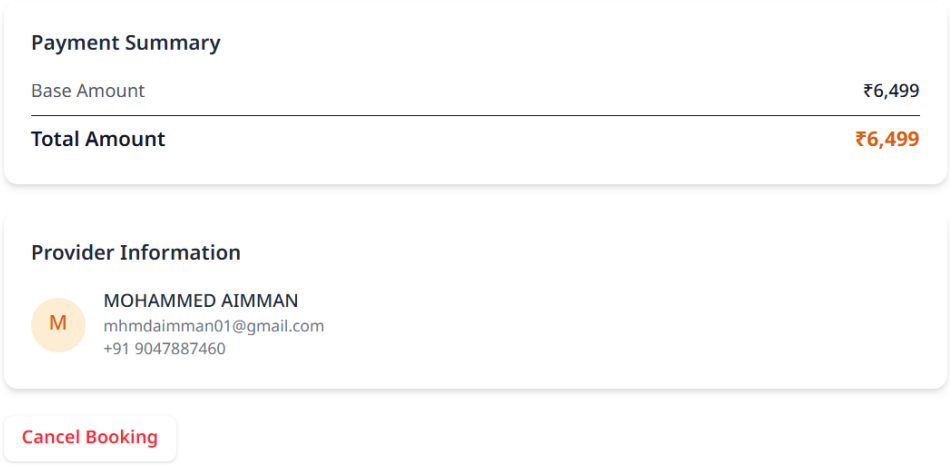


Fig 8.1.18 Payment Summary

This is the Booking Approval Panel or Order Management View for service providers, featuring customer details, payment summary, and primary action buttons to confirm or reject a pending request.

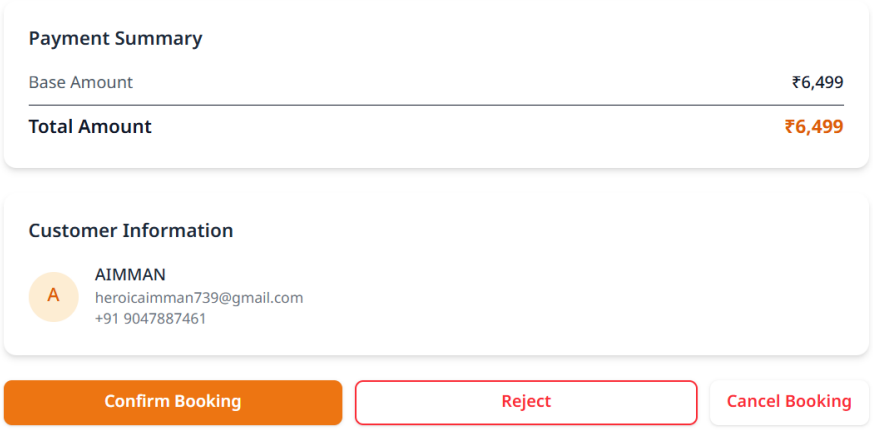


Fig 8.1.19 Booking Confirmation

This is the Payment Receipt and Feedback Submission Form, displayed after a service is completed to confirm the transaction and collect a user review.

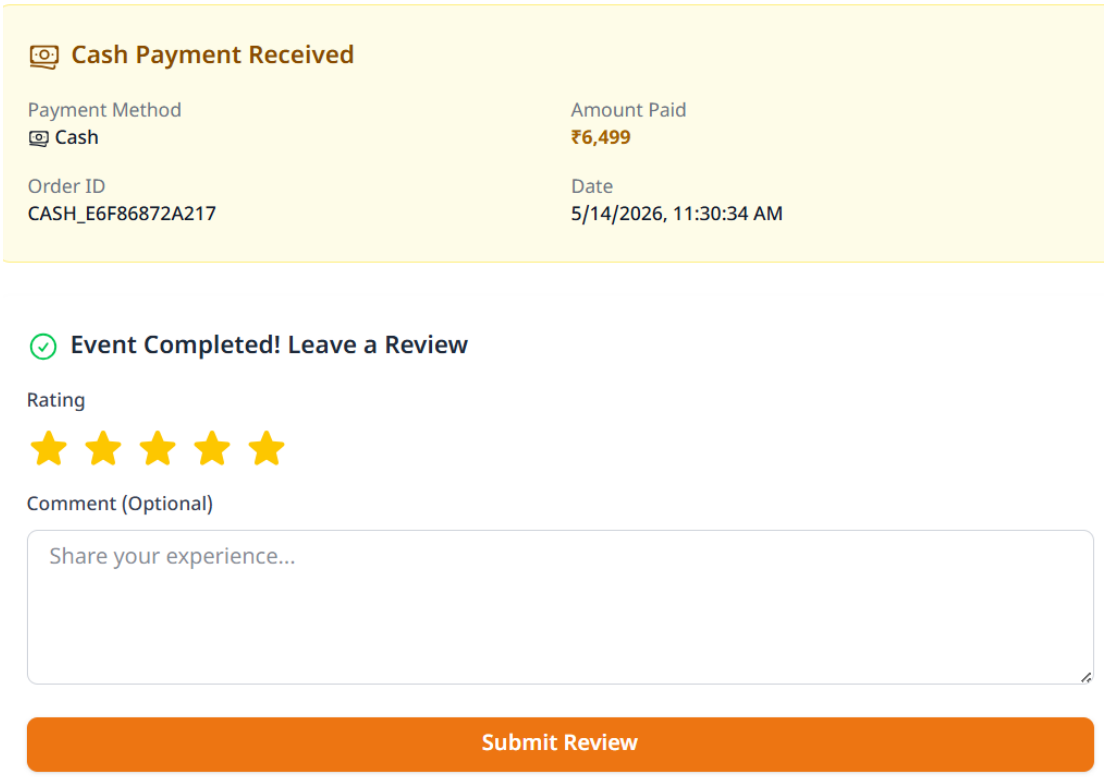


Fig 8.1.20 Receipt and Feedback Submission form

This is the User Bookings Dashboard or Order History View, where customers can track, filter by status (using the tabbed navigation), and manage all their past and upcoming service reservations.

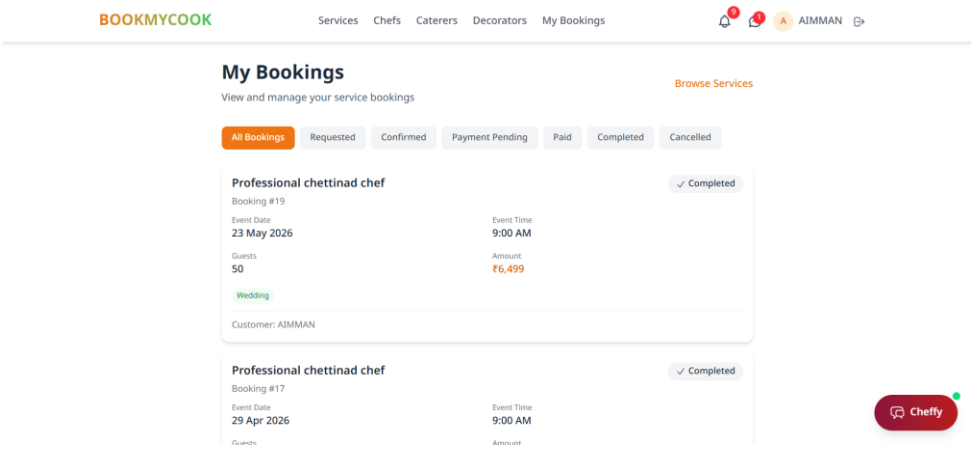


Fig 8.1.21 Booking History

Provider's Dashboard that displays all activity about the provider .

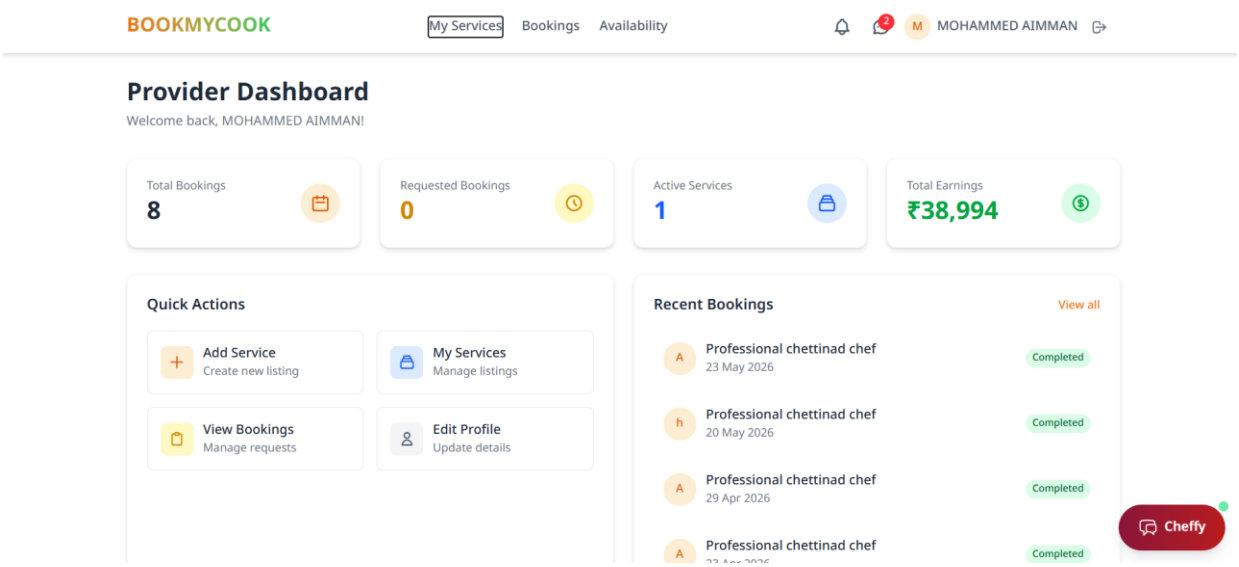


Fig 8.1.22 Provider Dashboard

This is the Service Management Dashboard or Provider Listing View, where chefs and vendors can add, edit, and manage their active service offerings.

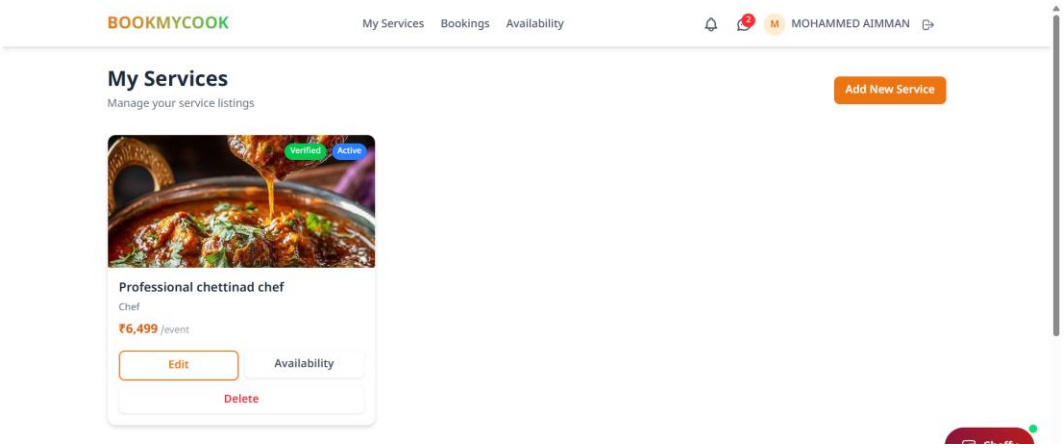


Fig 8.1.23 MyServices

This is the broad term for the entire page where vendors or chefs configure their working hours and open booking slots.

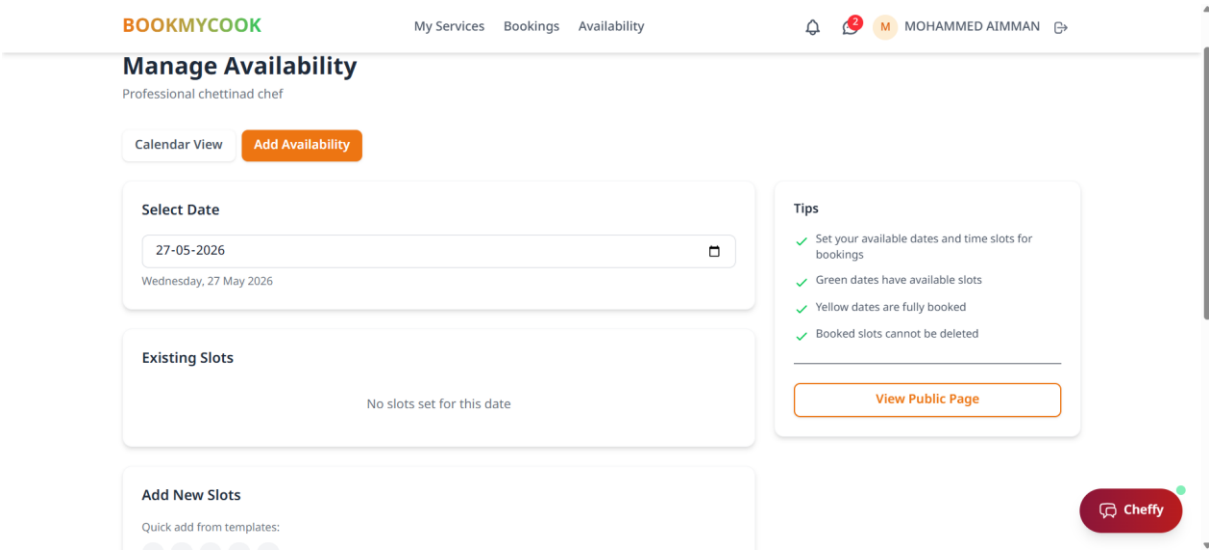


Fig 8.1.24 Manage availability

This is the most standard industry term. It refers to the restricted area of the application where administrators can view overarching system data, manage users, and monitor platform health.

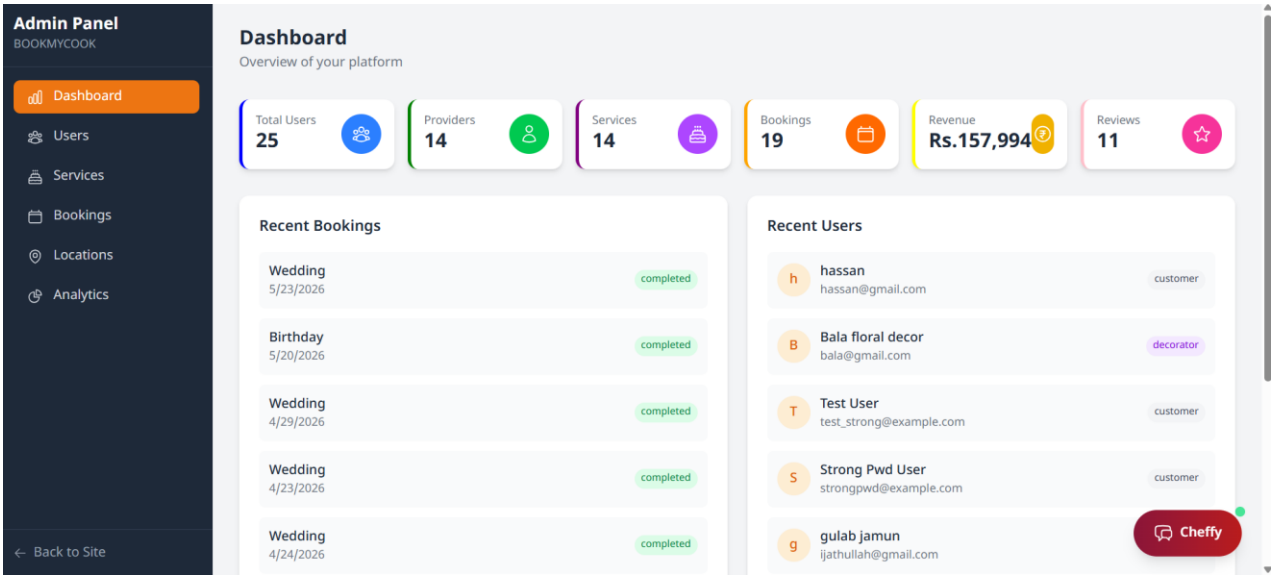


Fig 8.1.25 Admin Dashboard

This is the standard term for the entire page where admins can view, search, and manage all accounts (customers, chefs, decorators) registered on the platform.

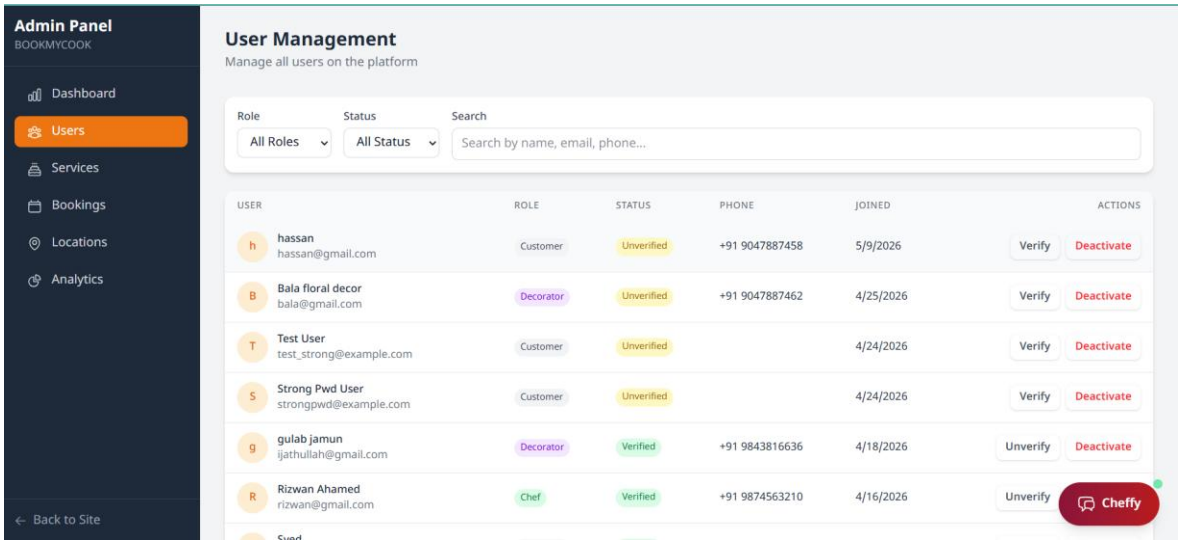


Fig 8.1.26 Admin User Management

From a functional backend perspective, developers might refer to this as a taxonomy or lookup table manager, as it manages the hierarchical data (Cities -> Areas) used throughout the rest of the application.

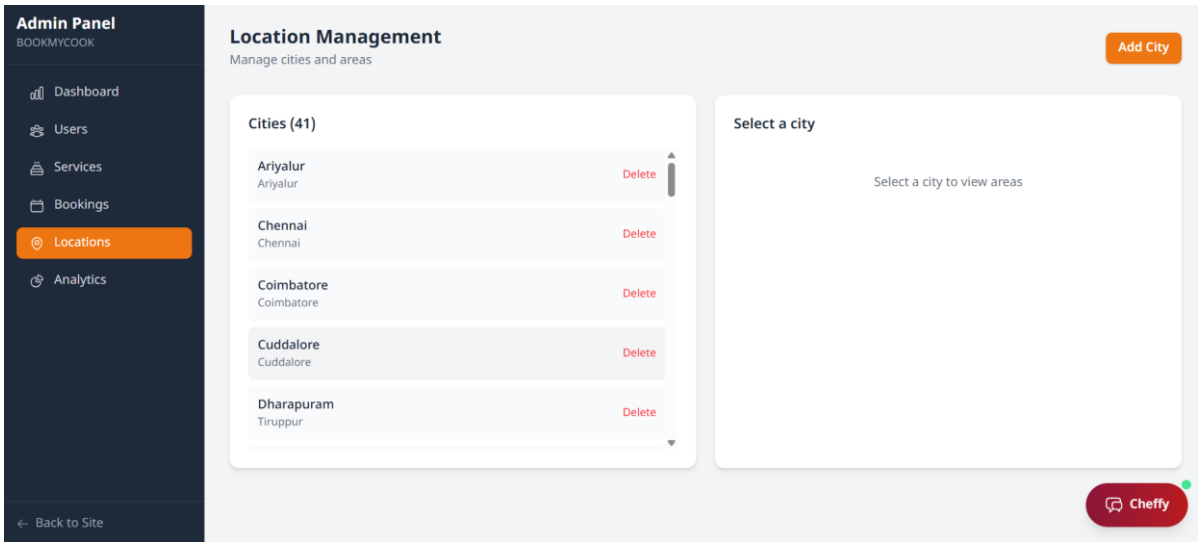


Fig 8.1.27 Location Management

CHAPTER – 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 CONCLUSION

In conclusion, the "Book My Cook" platform successfully addresses the systemic inefficiencies prevalent in the traditional catering and event management industry by delivering a unified, AI-enhanced digital marketplace. By seamlessly integrating a modern technology stack—featuring a responsive React.js frontend, a robust Python FastAPI backend, and reliable PostgreSQL database management—the project transforms a fragmented, manual coordination process into a streamlined and transparent ecosystem. The incorporation of advanced features, such as the natural language-processing "Cheffy" AI assistant, rule-based heuristic recommendation algorithms, real-time WebSocket communication, and secure Razorpay financial transactions, ensures a superior and frictionless experience for all stakeholders. Ultimately, this platform not only empowers local culinary professionals by expanding their digital footprint and safeguarding their revenues through secure booking protocols, but it also provides customers with unprecedented reliability and ease in event planning. "Book My Cook" establishes a new operational standard for the regional catering sector, proving that the strategic application of web technologies and artificial intelligence can successfully modernize even the most traditional service industries.

9.2 FUTURE ENHANCEMENTS

While "Book My Cook" provides a robust foundational ecosystem for culinary service booking, several avenues exist for future expansion to further enrich the platform’s utility. A primary focus for future development is the integration of advanced Machine Learning models to analyze historical booking data and user feedback, enabling hyper-personalized service recommendations that predict customer preferences before they search.

The platform can be expanded to include integrated supply chain logistics, allowing cooks to order raw materials and ingredients directly through partnered vendors within the dashboard. Furthermore, the introduction of a smart contract-based payment system utilizing blockchain technology could offer decentralized, tamper-proof financial agreements and instantaneous, feeless micro-transactions.

We also plan to expand the AI assistant, "Cheffy," with voice recognition capabilities and multilingual support to improve accessibility for a wider demographic across different regions. Finally, the development of dedicated native mobile applications for both iOS and Android platforms will significantly enhance the user experience by leveraging device-specific features like push notifications and GPS-based real-time tracking of service providers.

CHAPTER – 10

REFERENCES

[1] J. Smith *et al*., "Digital transformation in service booking platforms: Enhancing user experience, scalability, and security," *IEEE Access*, vol. 11, pp. 45210–45225, 2023.

[2] P. Sharma *et al*., "Role-based access control and secure authentication mechanisms for multi-user web applications," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 3112–3124, 2023.

[3] M. Johnson, "Recommendation systems for service discovery in e-commerce platforms," *IEEE Access*, vol. 10, pp. 77421–77436, 2022.

[4] D. Chen, "AI-powered chatbots for customer assistance in online platforms," *IEEE Transactions on Artificial Intelligence*, vol. 2, no. 3, pp. 210–221, 2021.

[5] S. Williams, "Secure payment integration in web applications using Razorpay," in *Proc. IEEE Int. Conf. on FinTech and Digital Payments*, 2023, pp. 88–95.

[6] R. Brown, "Enterprise security measures for web application protection," *IEEE Access*, vol. 10, pp. 99821–99837, 2022.

[7] E. Davis, "Real-time messaging systems for service provider communication," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4210–4221, 2021.

[8] J. Wilson, "PostgreSQL database design for scalable web applications," *IEEE Software*, vol. 40, no. 2, pp. 74–82, 2023.

[9] L. Anderson, "React.js frontend development for single-page applications," *IEEE Internet Computing*, vol. 26, no. 5, pp. 55–63, 2022.

[10] K. Rajesh, "Review and rating systems for building trust in online marketplaces," *IEEE Access*, vol. 9, pp. 120441–120455, 2021.

[11] T. Martinez, “Administrative dashboard design for platform management,” in *Proc. IEEE Int. Conf. on Smart Computing and Informatics*, 2022, pp. 332–339.

[12] A. Gupta, “Location-based services for regional marketplace platforms,” *IEEE Transactions on Mobile Computing*, vol. 20, no. 11, pp. 3255–3267, 2021.

[13] C. Lee, “Service availability management and calendar integration for online booking systems,” *IEEE Access*, vol. 11, pp. 66712–66728, 2023.

[14] D. Harris, “RESTful API design and documentation for web applications,” *IEEE Software*, vol. 38, no. 6, pp. 91–99, 2021.

CHAPTER 11
SAMPLE CODE

SERVER

app.py

```
import os
from dotenv import load_dotenv
from app import create_app, db
load_dotenv()
app = create_app()
if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
```

init.py

```
import os
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
from flask_cors import CORS
from flask_jwt_extended import JWTManager
from dotenv import load_dotenv

load_dotenv()

db = SQLAlchemy()
migrate = Migrate()
jwt = JWTManager()

SECURITY_ENABLED = os.environ.get('SECURITY_ENABLED', 'true').lower() == 'true'

def create_app(config_name="default"):
    app = Flask(__name__)
```

```
app.config["SECRET_KEY"] = os.environ.get(
    "SECRET_KEY", "dev-secret-key-change-in-production"
)
database_url = os.environ.get("DATABASE_URL")
if not database_url:
    basedir = os.path.abspath(os.path.dirname(__file__))
    database_url = f"sqlite:///{os.path.join(basedir, '..', 'bookmycook.db')}"
app.config["SQLALCHEMY_DATABASE_URI"] = database_url
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
app.config["JWT_SECRET_KEY"] = os.environ.get(
    "JWT_SECRET_KEY", "jwt-secret-key-change-in-production"
)
app.config["JWT_ACCESS_TOKEN_EXPIRES"] = 24 * 60 * 60

db.init_app(app)
migrate.init_app(app, db)
CORS(app)
jwt.init_app(app)

if SECURITY_ENABLED:
    from app.utils.rate_limiter import init_rate_limiter
    init_rate_limiter(app)

from app.routes.auth import auth_bp
from app.routes.users import users_bp
from app.routes.locations import locations_bp
from app.routes.services import services_bp
from app.routes.upload import upload_bp
from app.routes.bookings import bookings_bp
from app.routes.availability import availability_bp
from app.routes.dishes import dishes_bp
from app.routes.history import history_bp
from app.routes.payments import payments_bp
from app.routes.reviews import reviews_bp
from app.routes.messages import messages_bp
```

```
from app.routes.notifications import notifications_bp
from app.routes.admin import admin_bp
from app.routes.chat import chat_bp
from app.routes.two_factor import two_factor_bp

app.register_blueprint(auth_bp, url_prefix="/api/auth")
app.register_blueprint(users_bp, url_prefix="/api/users")
app.register_blueprint(locations_bp, url_prefix="/api")
app.register_blueprint(services_bp, url_prefix="/api/services")
app.register_blueprint(upload_bp, url_prefix="/api/upload")
app.register_blueprint(bookings_bp, url_prefix="/api/bookings")
app.register_blueprint(availability_bp, url_prefix="/api/availability")
app.register_blueprint(dishes_bp, url_prefix="/api/dishes")
app.register_blueprint(history_bp, url_prefix="/api/history")
app.register_blueprint(payments_bp, url_prefix="/api/payments")
app.register_blueprint(reviews_bp, url_prefix="/api/reviews")
app.register_blueprint(messages_bp, url_prefix="/api/messages")
app.register_blueprint(notifications_bp, url_prefix="/api/notifications")
app.register_blueprint(admin_bp, url_prefix="/api/admin")
app.register_blueprint(chat_bp, url_prefix="/api/chat")
app.register_blueprint(two_factor_bp, url_prefix="/api/2fa")

upload_folder = os.path.join(os.path.dirname(os.path.dirname(__file__)), "uploads")
os.makedirs(os.path.join(upload_folder, "services"), exist_ok=True)
os.makedirs(os.path.join(upload_folder, "profiles"), exist_ok=True)

from flask import send_from_directory

@app.route("/uploads/<path:filename>")
def serve_upload(filename):
    return send_from_directory(upload_folder, filename)

@app.route("/api/health")
def health_check():
    return {"status": "healthy", "message": "BOOKMYCOOK API is running"}
```

```
@app.route("/favicon.ico")
def favicon():
    return "", 204

@app.cli.command("seed-db")
def seed_db():
    from app.models.location import City, Area
    from app.utils.seed_data import (
        TAMIL_NADU_CITIES,
        CHENNAI_AREAS,
        COIMBATORE_AREAS,
        MADURAI_AREAS,
        TRICHY_AREAS,
        SALEM_AREAS,
    )

    if City.query.first():
        print("Database already seeded.")
        return

    print("Seeding database...")

    city_map = {}
    for name, district in TAMIL_NADU_CITIES:
        city = City(name=name, district=district)
        db.session.add(city)
        city_map[name] = city

    db.session.commit()

    for name, pincode in CHENNAI_AREAS:
        area = Area(city_id=city_map["Chennai"].id, name=name, pincode=pincode)
        db.session.add(area)

    for name, pincode in COIMBATORE_AREAS:
        area = Area(city_id=city_map["Coimbatore"].id, name=name, pincode=pincode)
```

```
            db.session.add(area)

        for name, pincode in MADURAI_AREAS:
            area = Area(city_id=city_map["Madurai"].id, name=name, pincode=pincode)
            db.session.add(area)

        for name, pincode in TRICHY_AREAS:
            area = Area(
                city_id=city_map["Tiruchirappalli"].id, name=name, pincode=pincode
            )
            db.session.add(area)

        for name, pincode in SALEM_AREAS:
            area = Area(city_id=city_map["Salem"].id, name=name, pincode=pincode)
            db.session.add(area)

        db.session.commit()
        print("Database seeded successfully!")

    @app.cli.command("seed-dummy")
    def seed_dummy():
        from datetime import datetime, date, time
        import bcrypt
        from app.models.location import City, Area
        from app.models.user import User, Profile
        from app.models.service import Service
        from app.models.booking import Booking
        from app.models.signature_dish import SignatureDish
        from app.models.event_history import EventHistory
        from app.models.review import Review
        from app.models.availability import Availability
        from app.utils.seed_dummy_data import (
            DUMMY_USERS,
            CHEF_SERVICES,
            CATERER_SERVICES,
            DECORATOR_SERVICES,
```

```
    EVENT_HISTORY_DATA,
    SAMPLE_BOOKINGS,
    SAMPLE_REVIEWS,
    AVAILABILITY_DATA,
)

if User.query.first():
    print("Dummy data already exists. Skipping...")
    return

print("Seeding dummy data...")

city_map = {c.name: c for c in City.query.all()}
area_map = {}
for city in City.query.all():
    for area in Area.query.filter_by(city_id=city.id).all():
        area_map[f"{city.name}_{area.name}"] = area

users = {}
for user_data in DUMMY_USERS:
    password_hash = bcrypt.hashpw(
        "password123".encode("utf-8"), bcrypt.gensalt()
    ).decode("utf-8")
    user = User(
        email=user_data["email"],
        password_hash=password_hash,
        full_name=user_data["full_name"],
        phone=user_data["phone"],
        role=user_data["role"],
        is_verified=True,
        is_active=True,
    )
    db.session.add(user)
    db.session.flush()
    users[user_data["email"]] = user
```

```
            city = city_map.get(user_data["city"])
            area = area_map.get(f"{user_data['city']}_{user_data['area']}")
            profile = Profile(
                user_id=user.id,
                profile_image=user_data["profile_image"],
                bio=user_data["bio"],
                city_id=city.id if city else None,
                area_id=area.id if area else None,
            )
            db.session.add(profile)

        db.session.commit()
        print(f"Created {len(users)} users")

        services = []
                    all_service_data = CHEF_SERVICES + CATERER_SERVICES +
DECORATOR_SERVICES

        for idx, service_data in enumerate(all_service_data):
            user = users[service_data["user_email"]]
            city = city_map.get(
                user.profile.city.name
                if user.profile and user.profile.city
                else "Chennai"
            )
            area = None
            if user.profile and user.profile.area:
                area = area_map.get(
                    f"{user.profile.city.name}_{user.profile.area.name}"
                )

            service = Service(
                user_id=user.id,
                title=service_data["title"],
                description=service_data["description"],
                service_type=service_data["service_type"],
```

```
            experience_years=service_data["experience_years"],
            price_per_event=service_data["price_per_event"],
            serves_veg=service_data["serves_veg"],
            serves_non_veg=service_data["serves_non_veg"],
            min_guests=service_data["min_guests"],
            max_guests=service_data["max_guests"],
            city_id=city.id if city else None,
            area_id=area.id if area else None,
            is_active=True,
            is_verified=True,
        )
        service.set_cuisine_types(service_data["cuisine_types"])
        service.set_event_types(service_data["event_types"])
        service.set_images(service_data.get("images", []))

        db.session.add(service)
        db.session.flush()
        services.append(service)

        if "dishes" in service_data:
            for dish_idx, dish_data in enumerate(service_data["dishes"]):
                dish = SignatureDish(
                    service_id=service.id,
                    name=dish_data["name"],
                    description=dish_data["description"],
                    image_url=dish_data["image_url"],
                    cuisine_type=dish_data["cuisine_type"],
                    is_veg=dish_data["is_veg"],
                    display_order=dish_idx + 1,
                )
                db.session.add(dish)

    db.session.commit()
    print(
                f"Created {len(services)} services ({len(CHEF_SERVICES)} chefs,
{len(CATERER_SERVICES)} caterers, {len(DECORATOR_SERVICES)} decorators)"
```

```
)

for event_data in EVENT_HISTORY_DATA:
    service = services[event_data["service_index"]]
    event = EventHistory(
        service_id=service.id,
        event_date=datetime.strptime(
            event_data["event_date"], "%Y-%m-%d"
        ).date(),
        event_type=event_data["event_type"],
        number_of_guests=event_data["number_of_guests"],
        venue=event_data["venue"],
        customer_name=event_data["customer_name"],
        customer_testimonial=event_data["customer_testimonial"],
        is_featured=event_data["is_featured"],
    )
    event.set_cuisine_types(event_data["cuisine_types"])
    event.set_photos(event_data.get("photos", []))
    db.session.add(event)

db.session.commit()
print("Created event history")

bookings = []
for booking_data in SAMPLE_BOOKINGS:
    customer = users[booking_data["customer_email"]]
    service = services[booking_data["service_index"]]
    city = city_map.get(booking_data["city"])
    area = area_map.get(f"{booking_data['city']}_{booking_data['area']}")

    booking = Booking(
        service_id=service.id,
        customer_id=customer.id,
        provider_id=service.user_id,
        event_date=datetime.strptime(
            booking_data["event_date"], "%Y-%m-%d"
```

```
            ).date(),
            event_time=datetime.strptime(
                booking_data["event_time"], "%H:%M"
            ).time(),
            event_type=booking_data["event_type"],
            event_address=booking_data["event_address"],
            city_id=city.id if city else None,
            area_id=area.id if area else None,
            number_of_guests=booking_data["number_of_guests"],
            special_requirements=booking_data["special_requirements"],
            base_amount=booking_data["base_amount"],
            total_amount=booking_data["total_amount"],
            status=booking_data["status"],
        )
        db.session.add(booking)
        db.session.flush()
        bookings.append(booking)

    db.session.commit()
    print(f"Created {len(bookings)} bookings")

    for review_data in SAMPLE_REVIEWS:
        booking = bookings[review_data["booking_index"]]
        review = Review(
            booking_id=booking.id,
            service_id=booking.service_id,
            user_id=booking.customer_id,
            rating=review_data["rating"],
            comment=review_data["comment"],
            is_visible=True,
        )
        db.session.add(review)

        service = Service.query.get(booking.service_id)
        if service:
            service.total_reviews += 1
```

```
        total_rating = (
            db.session.query(db.func.sum(Review.rating))
            .filter_by(service_id=service.id, is_visible=True)
            .scalar()
            or 0
        )
        service.rating = round(total_rating / service.total_reviews, 2)

db.session.commit()
print("Created reviews")

for avail_data in AVAILABILITY_DATA:
    if avail_data["service_index"] < len(services):
        service = services[avail_data["service_index"]]
        avail = Availability(
            service_id=service.id,
            date=datetime.strptime(avail_data["date"], "%Y-%m-%d").date(),
            start_time=datetime.strptime(
                avail_data["start_time"], "%H:%M"
            ).time(),
            end_time=datetime.strptime(avail_data["end_time"], "%H:%M").time(),
            is_available=avail_data["is_available"],
        )
        db.session.add(avail)

db.session.commit()
print(f"Created {len(AVAILABILITY_DATA)} availability slots")

print("Dummy data seeded successfully!")
print("\nTest accounts (password: password123):")
for email in [
    "customer1@example.com",
    "chef1@example.com",
    "caterer1@example.com",
    "decorator1@example.com",
]:
```

```
            print(f"  - {email}")

    return app

]):
        return jsonify({"message": "Email, password, and full name are required"}), 400

    if SECURITY_ENABLED:
        is_strong, password_errors = validate_password(password)
        if not is_strong:
                return jsonify({"message": "Password does not meet requirements", "errors":
password_errors}), 400

    if User.query.filter_by(email=email).first():
        return jsonify({"message": "Email already registered"}), 400

    if phone and User.query.filter_by(phone=phone).first():
        return jsonify({"message": "Phone number already registered"}), 400

    valid_roles = ["customer", "chef", "caterer", "decorator"]
    if role not in valid_roles:
        return jsonify({"message": "Invalid role"}), 400

    password_hash = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode(
        "utf-8"
    )

    user = User(
        email=email,
        password_hash=password_hash,
        full_name=full_name,
        phone=phone,
        role=role,
    )

    db.session.add(user)
```

```
    db.session.flush()

    profile = Profile(user_id=user.id)
    db.session.add(profile)
    db.session.commit()

    access_token = create_access_token(identity=str(user.id))

    if SECURITY_ENABLED:
        log_auth_event(
            AuditEventType.REGISTER,
            user_id=user.id,
            email=email,
            success=True,
            details={"role": role}
        )
        create_session(user.id, access_token)

    return jsonify(
        {
            "message": "User registered successfully",
            "token": access_token,
            "user": user.to_dict(),
        }
    ), 201


@auth_bp.route("/login", methods=["POST"])
@limiter.limit(get_rate_limit("auth_login"))
def login():
```

DATABASE

Setup_Database.sh

```
#!/bin/bash

# BOOKMYCOOK PostgreSQL Setup Script
# Run with: sudo bash setup_database.sh

set -e

echo "=== Installing PostgreSQL ==="
apt update
apt install -y postgresql postgresql-contrib

echo "=== Starting PostgreSQL ==="
systemctl start postgresql
systemctl enable postgresql

echo "=== Creating database and user ==="
sudo -u postgres psql -c "CREATE DATABASE bookmycook;"
sudo -u postgres psql -c "CREATE USER bookmycook_user WITH PASSWORD 'bookmycook123';"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE bookmycook TO bookmycook_user;"
sudo -u postgres psql -c "ALTER USER bookmycook_user CREATEDB;"

echo "=== Running schema ==="
sudo -u postgres psql -d bookmycook -f /home/mhmdaimman/BOOKMYCOOK/database/schema.sql
sudo -u postgres psql -d bookmycook -f /home/mhmdaimman/BOOKMYCOOK/database/seeds/tamilnadu_cities.sql

echo "=== PostgreSQL Setup Complete ==="
echo "Database: bookmycook"
```

```
echo "User: bookmycook_user"
echo "Password: bookmycook123"
```

init_db.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app import create_app, db
from app.models.user import User, Profile
from app.models.location import City, Area
from app.models.service import Service
from app.models.booking import Booking
from app.models.availability import Availability
from app.models.signature_dish import SignatureDish
from app.models.event_history import EventHistory
from app.models.payment import Payment
from app.models.review import Review
from app.utils.audit import AuditLog
from app.utils.account_lockout import FailedLoginAttempt
from app.utils.totp import TwoFactorAuth
from app.utils.session_manager import UserSession
from app.utils.seed_data import (
    TAMIL_NADU_CITIES,
    CHENNAI_AREAS,
    COIMBATORE_AREAS,
    MADURAI_AREAS,
    TRICHY_AREAS,
    SALEM_AREAS,
)

app = create_app()

with app.app_context():
```

```
print("Creating tables...")
db.create_all()

if City.query.first():
    print("Database already seeded.")
else:
    print("Seeding Tamil Nadu cities...")

    city_map = {}
    for name, district in TAMIL_NADU_CITIES:
        city = City(name=name, district=district)
        db.session.add(city)
        city_map[name] = city

    db.session.commit()

    print("Seeding areas...")

    for name, pincode in CHENNAI_AREAS:
        area = Area(city_id=city_map["Chennai"].id, name=name, pincode=pincode)
        db.session.add(area)

    for name, pincode in COIMBATORE_AREAS:
        area = Area(city_id=city_map["Coimbatore"].id, name=name, pincode=pincode)
        db.session.add(area)

    for name, pincode in MADURAI_AREAS:
        area = Area(city_id=city_map["Madurai"].id, name=name, pincode=pincode)
        db.session.add(area)

    for name, pincode in TRICHY_AREAS:
        area = Area(
            city_id=city_map["Tiruchirappalli"].id, name=name, pincode=pincode
        )
        db.session.add(area)
```

```
        for name, pincode in SALEM_AREAS:
            area = Area(city_id=city_map["Salem"].id, name=name, pincode=pincode)
            db.session.add(area)

        db.session.commit()
        print("Database seeded successfully!")

    print("\nDatabase initialization complete!")
    print(f"Total cities: {City.query.count()}")
    print(f"Total areas: {Area.query.count()}")
```

FRONTEND

Home.jsx

```
import { Link, useNavigate } from 'react-router-dom';
import { useState } from 'react';
import { UserGroupIcon, CakeIcon, SparklesIcon } from '@heroicons/react/24/outline';
import Button from '../components/common/Button';
import HomeNavbar from '../components/common/HomeNavbar';
import Footer from '../components/layout/Footer';
import { useAuth } from '../context/AuthContext';

const Home = () => {
  const { isAuthenticated } = useAuth();
  const navigate = useNavigate();
  const [searchQuery, setSearchQuery] = useState('');
  const [imageLoaded, setImageLoaded] = useState(false);
  const [imageError, setImageError] = useState(false);

  const features = [
    {
      title: 'Professional Chefs',
       description: 'Hire experienced chefs specializing in Tamil cuisine, Chettinad, Kongu, and more.',
      icon: UserGroupIcon,
```

```
    link: '/chefs',
  },
  {
    title: 'Catering Services',
      description: 'Complete catering solutions for weddings, corporate events, and family functions.',
    icon: CakeIcon,
    link: '/caterers',
  },
  {
    title: 'Decoration Services',
    description: 'Transform your venue with stunning decorations for any occasion.',
    icon: SparklesIcon,
    link: '/decorators',
  },
 ];

 const whyChooseUs = [
  {
    title: 'Verified Providers',
    description: 'All chefs and caterers are verified with background checks and valid certifications across Tamil Nadu.',
    icon: (
      <svg className="w-12 h-12 text-white" fill="none" stroke="currentColor" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={1.5} d="M9 12.75L11.25 15 15 9.75m-3-7.036A11.959 11.959 0 013.598 6 11.99 11.99 0 003 9.749c0 5.592 3.824 10.29 9 11.623 5.176-1.332 9-6.03 9-11.622 0-1.31-.21-2.571-.598-3.751h-.152c-3.196 0-6.1-1.488-8-3.864z" />
      </svg>
    ),
  },
  {
    title: 'Tamil Nadu Focus',
    description: 'Specialized in Chettinad, Kongu, and traditional Tamil cuisine with deep local expertise.',
```

```
    icon: (
      <svg className="w-12 h-12 text-white" fill="none" stroke="currentColor" viewBox="0 0
24 24">
       <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={1.5} d="M15 10.5a3
3 0 11-6 0 3 3 0 016 0z" />
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={1.5} d="M19.5 10.5c0
7.142-7.5 11.25-7.5 11.25S4.5 17.642 4.5 10.5a7.5 7.5 0 1115 0z" />
     </svg>
    ),
  },
  {
    title: 'Secure Payments',
     description: 'Safe online and cash payment options with transparent pricing and no hidden
fees.',
    icon: (
      <svg className="w-12 h-12 text-white" fill="none" stroke="currentColor" viewBox="0 0
24 24">
          <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={1.5} d="M2.25
8.25h19.5M2.25 9h19.5m-16.5 5.25h6m-6 2.25h3m-3.75 3h15a2.25 2.25 0 002.25-
2.25V6.75A2.25 2.25 0 0019.5 4.5h-15a2.25 2.25 0 00-2.25 2.25v10.5A2.25 2.25 0 004.5 19.5z"
/>
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={1.5} d="M16.5 14.5l2
2 4-4" />
     </svg>
    ),
  },
  {
    title: '24/7 Support',
     description: 'Dedicated support team available round the clock for all your event planning
needs.',
    icon: (
      <svg className="w-12 h-12 text-white" fill="none" stroke="currentColor" viewBox="0 0
24 24">
          <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={1.5} d="M8.625
12a.375.375 0 11-.75 0 .375.375 0 01.75 0zm0 0H8.25m4.125 0a.375.375 0 11-.75 0 .375.375 0
01.75 0zm0 0H12m4.125 0a.375.375 0 11-.75 0 .375.375 0 01.75 0zm0 0h-.375M21 12c0 4.556-
```

```
4.03 8.25-9 8.25a9.764 9.764 0 01-2.555-.337A5.972 5.972 0 015.41 20.97a5.969 5.969 0 01-.474-.065 4.48 4.48 0 00.978-2.025c.09-.197.175-.394.254-.59A8.22 8.22 0 013 12c0-4.556 4.03-8.25 9-8.25s9 3.694 9 8.25z" />
      </svg>
    ),
  },
];

const popularCities = [
  'Chennai', 'Coimbatore', 'Madurai', 'Trichy', 'Salem', 'Tirunelveli'
];

const stats = [
  { value: '500+', label: 'Service Providers' },
  { value: '1000+', label: 'Events Completed' },
  { value: '40+', label: 'Cities Covered' },
  { value: '4.8', label: 'Average Rating' },
];

const handleSearch = (e) => {
  e.preventDefault();
  if (searchQuery.trim()) {
    navigate(`/services?search=${encodeURIComponent(searchQuery)}`);
  }
};

return (
  <div className="min-h-screen">
    {/* Home Navbar */}
    {!isAuthenticated && <HomeNavbar />}

    {/* Hero Section - Full Screen with Overlay */}
    <section className="relative min-h-screen flex items-center justify-center">
      {/* Fallback Gradient Background */}
        <div className="absolute inset-0 bg-gradient-to-br from-[#8B1538] via-[#A31B47] to-[#B91C1C]" />
```

```
      {/* Unsplash Background Image */}
      {!imageError && (
       <img
        src="https://images.unsplash.com/photo-1585937421612-70a008356fbe?w=1920&q=80"
        alt="South Indian cuisine background"
         className={`absolute inset-0 w-full h-full object-cover transition-opacity duration-700
${
          imageLoaded ? 'opacity-100' : 'opacity-0'
        }`}
        onLoad={() => setImageLoaded(true)}
        onError={() => setImageError(true)}
       />
      )}

      {/* Dark Overlay */}
      <div className="absolute inset-0 bg-black/60" />

      {/* Hero Content */}
      <div className="relative z-10 text-center px-4 sm:px-6 lg:px-8 max-w-5xl mx-auto">
       {/* Logo/Brand */}
       <div className="mb-6">
        <span className="text-4xl md:text-5xl font-bold text-[#F59E0B]">BOOK</span>
        <span className="text-4xl md:text-5xl font-bold text-white">MYCOOK</span>
       </div>

       {/* Tagline */}
       <h1 className="text-3xl md:text-5xl lg:text-6xl font-bold text-white mb-6 leading-tight">
        Book the Best Services
        <br />
        <span className="text-[#F59E0B]">for Your Events</span>
       </h1>

       {/* Description */}
       <p className="text-lg md:text-xl text-gray-200 mb-10 max-w-2xl mx-auto">
```

```
        Tamil Nadu's trusted platform for hiring professional chefs, catering services, and decoration management for all your events.
      </p>

      {/* Search Bar */}
      <form onSubmit={handleSearch} className="mb-10 max-w-2xl mx-auto">
        <div className="flex flex-col sm:flex-row gap-3 bg-white/10 backdrop-blur-sm p-2 rounded-2xl">
          <div className="relative flex-1">
            <input
              type="text"
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
              placeholder="Search for chefs, caterers, decorators..."
                  className="w-full px-6 py-4 rounded-xl text-gray-800 placeholder-gray-500 focus:outline-none focus:ring-2 focus:ring-[#F59E0B]"
            />
          </div>
          <button
            type="submit"
              className="px-8 py-4 bg-[#F59E0B] hover:bg-[#D97706] text-white font-semibold rounded-xl transition-colors flex items-center justify-center gap-2"
          >
            <svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" />
            </svg>
            Search
          </button>
        </div>
      </form>

      {/* CTA Buttons */}
      <div className="flex flex-col sm:flex-row gap-4 justify-center">
        <Link to="/services">
          <Button
```

```
            variant="primary"
            size="lg"
            className="bg-[#F59E0B] hover:bg-[#D97706] text-white px-8 py-4 rounded-xl font-semibold"
          >
            Explore Services
          </Button>
        </Link>
        {!isAuthenticated && (
          <Link to="/register">
            <Button
              variant="outline"
              size="lg"
              className="border-2 border-white text-white hover:bg-white hover:text-[#8B1538] px-8 py-4 rounded-xl font-semibold"
            >
              Get Started
            </Button>
          </Link>
        )}
      </div>
    </div>

    {/* Scroll Indicator */}
    <div className="absolute bottom-8 left-1/2 -translate-x-1/2 animate-bounce z-10">
      <svg className="w-6 h-6 text-white/70" fill="none" stroke="currentColor" viewBox="0 0 24 24">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M19 14l-7 7m0 0l-7-7m7 7V3" />
      </svg>
    </div>
  </section>

  {/* Service Categories Section */}
  <section className="py-20 bg-[#FFFBEB]">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
```

```
        <div className="text-center mb-16">
          <h2 className="text-3xl md:text-4xl font-bold text-[#451A03] mb-4">
            Our Services
          </h2>
          <p className="text-lg text-[#78350F] max-w-2xl mx-auto">
            Find the perfect service provider for your next event. From intimate gatherings to grand celebrations.
          </p>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
          {features.map((feature, index) => {
            const IconComponent = feature.icon;
            return (
              <Link to={feature.link} key={index} className="group">
                <div className="bg-white rounded-2xl shadow-lg p-8 text-center h-full transition-all duration-300 group-hover:shadow-2xl group-hover:-translate-y-2 border-2 border-transparent group-hover:border-[#F59E0B]">
                  <div className="w-20 h-20 mx-auto mb-6 bg-gradient-to-br from-[#8B1538] to-[#B91C1C] rounded-2xl flex items-center justify-center shadow-lg">
                    <IconComponent className="w-10 h-10 text-white" />
                  </div>
                  <h3 className="text-xl font-bold text-[#451A03] mb-3">{feature.title}</h3>
                  <p className="text-[#78350F]">{feature.description}</p>
                </div>
              </Link>
            );
          })}
        </div>
      </div>
    </section>

    {/* Statistics Section */}
    <section className="py-16 bg-gradient-to-r from-[#8B1538] to-[#B91C1C]">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="grid grid-cols-2 md:grid-cols-4 gap-8">
```

```
          {stats.map((stat, index) => (
            <div key={index} className="text-center">
              <p className="text-4xl md:text-5xl font-bold text-[#F59E0B] mb-2">{stat.value}</p>
              <p className="text-white/80 text-sm md:text-base">{stat.label}</p>
            </div>
          ))}
        </div>
      </div>
    </section>

    {/* Why Choose Us Section */}
    <section className="py-20 bg-gradient-to-b from-white to-[#FFFBEB]">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="text-center mb-16">
          <h2 className="text-3xl md:text-4xl font-bold text-[#451A03] mb-4">
            Why Choose BOOKMYCOOK?
          </h2>
          <p className="text-lg text-[#78350F] max-w-2xl mx-auto">
            Tamil Nadu's most trusted platform for professional event services
          </p>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-8">
          {whyChooseUs.map((item, index) => (
            <div
              key={index}
                  className="group bg-white rounded-2xl shadow-lg p-8 text-center transition-all duration-300 hover:shadow-2xl hover:-translate-y-2 border-2 border-transparent hover:border-[#F59E0B]"
              >
                  <div className="w-24 h-24 mx-auto mb-6 bg-gradient-to-br from-[#8B1538] to-[#B91C1C] rounded-full flex items-center justify-center shadow-lg group-hover:animate-float transition-all duration-300">
                    {item.icon}
                  </div>
                  <h3 className="text-xl font-bold text-[#451A03] mb-3">{item.title}</h3>
```

```
            <p className="text-[#78350F] text-sm leading-relaxed">{item.description}</p>
          </div>
        ))}
      </div>
    </div>
  </section>

  {/* Popular Cities Section */}
  <section className="py-20 bg-white">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="text-center mb-12">
        <h2 className="text-3xl md:text-4xl font-bold text-[#451A03] mb-4">
          Popular Cities in Tamil Nadu
        </h2>
        <p className="text-lg text-[#78350F]">
          Find service providers in your city
        </p>
      </div>

      <div className="flex flex-wrap justify-center gap-4">
        {popularCities.map((city, index) => (
          <Link
            key={index}
            to={`/services?city=${city.toLowerCase()}`}
                 className="px-8 py-4 bg-[#FEF3C7] rounded-full shadow-md hover:shadow-lg transition-all text-[#451A03] font-semibold border-2 border-[#F59E0B]/30 hover:border-[#F59E0B] hover:bg-[#F59E0B] hover:text-white"
          >
            {city}
          </Link>
        ))}
      </div>
    </div>
  </section>

  {/* Provider CTA Section */}
```

```
      <section className="py-20 bg-[#451A03]">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex flex-col md:flex-row items-center justify-between gap-8">
            <div className="text-center md:text-left">
              <h2 className="text-3xl md:text-4xl font-bold text-white mb-4">
                Are You a Service Provider?
              </h2>
              <p className="text-xl text-[#F59E0B] max-w-lg">
                Join our platform and reach thousands of customers across Tamil Nadu.
              </p>
            </div>
            <Link to="/register?role=provider">
              <Button
                className="bg-[#F59E0B] hover:bg-[#D97706] text-white px-10 py-5 rounded-xl font-
bold text-lg whitespace-nowrap"
              >
                Register as Provider
              </Button>
            </Link>
          </div>
        </div>
      </section>

      <Footer />
    </div>
  );
};

export default Home;
```

Login.jsx

```
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import Input from '../components/common/Input';
```

```
import Button from '../components/common/Button';
import { EyeIcon, EyeSlashIcon } from '@heroicons/react/24/outline';

const Login = () => {
  const navigate = useNavigate();
  const { login } = useAuth();
  const [formData, setFormData] = useState({
    email: '',
    password: '',
  });
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value,
    }));
    if (errors[name]) {
      setErrors((prev) => ({
        ...prev,
        [name]: '',
      }));
    }
  };

  const validateForm = () => {
    const newErrors = {};
    if (!formData.email) {
      newErrors.email = 'Email is required';
    } else if (!/\S+@\S+\.\S+/.test(formData.email)) {
      newErrors.email = 'Invalid email format';
    }
    if (!formData.password) {
```

```
    newErrors.password = 'Password is required';
   }
   setErrors(newErrors);
   return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = async (e) => {
   e.preventDefault();
   if (!validateForm()) return;

   setLoading(true);
   const result = await login(formData.email, formData.password);
   setLoading(false);

   if (result.success) {
    navigate('/dashboard');
   } else {
    setErrors({ general: result.error });
   }
  };

  return (
   <div className="min-h-screen flex items-center justify-center bg-gray-50 py-12 px-4 sm:px-
6 lg:px-8">
    <div className="max-w-md w-full">
     <div className="text-center mb-8">
      <h1 className="text-3xl font-bold text-gradient">BOOKMYCOOK</h1>
      <h2 className="mt-4 text-2xl font-semibold text-gray-800">Welcome back</h2>
      <p className="mt-2 text-gray-600">Sign in to your account</p>
     </div>

     <form onSubmit={handleSubmit} className="bg-white rounded-xl shadow-md p-8">
      {errors.general && (
       <div className="mb-4 p-3 bg-red-100 text-red-700 rounded-lg">
        {errors.general}
       </div>
```

```
        )}

        <Input
          label="Email"
          type="email"
          name="email"
          value={formData.email}
          onChange={handleChange}
          error={errors.email}
          placeholder="Enter your email"
          required
        />

        <div className="mb-4">
          <label className="block text-sm font-medium text-gray-700 mb-1">Password</label>
          <div className="relative">
            <input
              type={showPassword ? 'text' : 'password'}
              name="password"
              value={formData.password}
              onChange={handleChange}
             className={`w-full px-4 py-2 pr-10 border rounded-lg focus:ring-2 focus:ring-primary-
500 focus:border-transparent outline-none ${
                errors.password ? 'border-red-500' : 'border-gray-300'
              }`}
              placeholder="Enter your password"
            />
            <button
              type="button"
              onClick={() => setShowPassword(!showPassword)}
             className="absolute right-3 top-1/2 -translate-y-1/2 text-gray-400 hover:text-gray-600"
            >
              {showPassword ? (
                <EyeSlashIcon className="w-5 h-5" />
              ) : (
                <EyeIcon className="w-5 h-5" />
```

```
            )}
          </button>
        </div>
        {errors.password && (
          <p className="mt-1 text-sm text-red-500">{errors.password}</p>
        )}
      </div>

      <div className="flex items-center justify-between mb-6">
        <label className="flex items-center">
          <input type="checkbox" className="rounded border-gray-300 text-primary-500 focus:ring-primary-500" />
          <span className="ml-2 text-sm text-gray-600">Remember me</span>
        </label>
        <Link to="/forgot-password" className="text-sm text-primary-500 hover:text-primary-600">
          Forgot password?
        </Link>
      </div>

      <Button
        type="submit"
        variant="primary"
        className="w-full"
        loading={loading}
      >
        Sign In
      </Button>

      <p className="mt-6 text-center text-gray-600">
        Don't have an account?{' '}
        <Link to="/register" className="text-primary-500 hover:text-primary-600 font-medium">
          Register
        </Link>
      </p>
```

```
        </form>
      </div>
    </div>
  );
};

export default Login;
```

Booking.jsx

```
import { useState, useEffect } from 'react';
import { useSearchParams, Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import BookingList from '../components/bookings/BookingList';
import { bookingAPI } from '../services/api';

const Bookings = () => {
  const { user, isProvider } = useAuth();
  const [searchParams, setSearchParams] = useSearchParams();
  const [bookings, setBookings] = useState([]);
  const [loading, setLoading] = useState(true);
  const [pagination, setPagination] = useState({
    total: 0,
    pages: 0,
    current_page: 1,
    has_next: false,
    has_prev: false,
  });

  const statusFilter = searchParams.get('status') || '';
  const page = searchParams.get('page') || 1;

  useEffect(() => {
    loadBookings();
  }, [statusFilter, page]);
```

```
const loadBookings = async () => {
  setLoading(true);
  try {
    const params = {
      page,
      per_page: 10,
      status: statusFilter || undefined,
      role: isProvider ? 'provider' : undefined,
    };

    const response = await bookingAPI.getAll(params);
    setBookings(response.data.bookings);
    setPagination({
      total: response.data.total,
      pages: response.data.pages,
      current_page: response.data.current_page,
      has_next: response.data.has_next,
      has_prev: response.data.has_prev,
    });
  } catch (error) {
    console.error('Failed to load bookings:', error);
  } finally {
    setLoading(false);
  }
};

const handleStatusChange = (status) => {
  setSearchParams({ status, page: 1 });
};

const handleConfirm = async (bookingId) => {
  try {
    await bookingAPI.confirm(bookingId);
    loadBookings();
  } catch (error) {
    console.error('Failed to confirm booking:', error);
```

```
  }
};

const handleReject = async (bookingId) => {
  if (!window.confirm('Are you sure you want to reject this booking?')) return;
  try {
    await bookingAPI.reject(bookingId);
    loadBookings();
  } catch (error) {
    console.error('Failed to reject booking:', error);
  }
};

const handleCancel = async (bookingId) => {
  if (!window.confirm('Are you sure you want to cancel this booking?')) return;
  try {
    await bookingAPI.cancel(bookingId);
    loadBookings();
  } catch (error) {
    console.error('Failed to cancel booking:', error);
  }
};

return (
  <div className="max-w-4xl mx-auto py-8 px-4 sm:px-6 lg:px-8">
    <div className="flex items-center justify-between mb-8">
      <div>
        <h1 className="text-3xl font-bold text-gray-800">
          {isProvider ? 'Booking Requests' : 'My Bookings'}
        </h1>
        <p className="text-gray-600 mt-2">
          {isProvider
            ? 'Manage booking requests from customers'
            : 'View and manage your service bookings'}
        </p>
      </div>
```

```
        <Link
          to="/services"
          className="text-primary-500 hover:text-primary-600 font-medium"
        >
          Browse Services
        </Link>
      </div>

      <BookingList
        bookings={bookings}
        loading={loading}
        statusFilter={statusFilter}
        onStatusChange={handleStatusChange}
        showActions={isProvider}
        onConfirm={handleConfirm}
        onReject={handleReject}
        onCancel={handleCancel}
      />

      {pagination.pages > 1 && (
        <div className="flex justify-center mt-8 gap-2">
          <button
            onClick={() => setSearchParams({ status: statusFilter, page: pagination.current_page - 1
})}
            disabled={!pagination.has_prev}
            className="px-4 py-2 border rounded-lg disabled:opacity-50 disabled:cursor-not-allowed
hover:bg-gray-50"
          >
            Previous
          </button>
          <span className="px-4 py-2 text-gray-600">
            Page {pagination.current_page} of {pagination.pages}
          </span>
          <button
            onClick={() => setSearchParams({ status: statusFilter, page: pagination.current_page + 1
})}
```

```
            disabled={!pagination.has_next}
            className="px-4 py-2 border rounded-lg disabled:opacity-50 disabled:cursor-not-allowed
hover:bg-gray-50"
          >
            Next
          </button>
        </div>
      )}
    </div>
  );
};
export default Bookings;
```

CHAPTER 12
LIST OF PUBLICATION

1. MOHAMMED AIMMAN M

MOHAMMED AIMMAN M, DR.R.KAVIARASI, Project work "BOOK MY COOK – An Online Catering and Contract Cooking Service Application", Proceedings of the of the International Conference On Emerging Innovation in science, Engineering and Technology (ICEISET – 2026), was held On 30th April 2026 Online,ISBS:9788169515-8,PP :553-538.


INTERNATIONAL CONFERENCE ON EMERGING INNOVATIONS IN SCIENCE, ENGINEERING AND TECHNOLOGY (ICEISET - 2026)
PROCEEDINGS

BOOK MY COOK – An Online chef Booking Service

Mohammed Aimman M
II MCA, Reg. No. 810024622031
Department of Computer Applications, University College of Engineering, BIT Campus
Anna University, Tiruchirappalli
Guide: Dr. R. Kaviarasi, Assistant Professor , Department of Computer Applications

ABSTRACT

"Book My Cook" is a centralized web application designed to automate catering, home cooking, and event decoration bookings, replacing inefficient manual processes. The platform features a React.js frontend and a Python Flask backend, communicating via secure RESTful APIs. Customer booking requirements are validated and securely stored in a PgSQL database. To ensure robust, future-proof security, sensitive user and booking data are protected using Post-Quantum Cryptography (PQC) during transmission and storage.

Validated requests are routed in real time to service providers, who can seamlessly accept or reject them via their dashboard, triggering dynamic status updates for the customer. Additionally, the system leverages Machine Learning algorithms to analyze historical data and user behavior, providing intelligent service recommendations. This end-to-end workflow minimizes manual intervention, enhances data security, ensures operational transparency, and delivers a highly efficient booking experience for both customers and service providers.

Keywords: *Event Service Booking, AI-Powered Recommendations, Razorpay Payment Integration, JWT Authentication, PostgreSQL, React JS, Flask, SQLAlchemy, Two-Factor Authentication, Rate Limiting, Tamil Nadu Market, Service Marketplace, Real-Time Messaging, Booking Management System, Provider Dashboard, Admin Analytics, Audit Logging, Account Security, Service Discovery, Review System, Location-Based Filtering*

I. INTRODUCTION

The event services industry in Tamil Nadu operates through fragmented channels, forcing customers to rely on word-of-mouth referrals, social media groups, and offline directories to find professional service providers. This traditional approach creates significant inefficiencies, including lack of pricing transparency, difficulty in verifying provider credibility, and time-consuming manual coordination that often results in scheduling conflicts and miscommunication.

The rapid growth of the event management sector has further complicated service discovery. Customers planning weddings, corporate events, or social gatherings require fundamentally different service categories—professional chefs for personalized culinary experiences, catering services for large-scale events, or decorators for venue aesthetics—yet must navigate multiple disconnected platforms to find each service type, with no unified comparison or booking mechanism.

Modern web technologies offer a transformative solution to these challenges. Digital platforms can aggregate verified service providers, enable real-time availability management, and facilitate secure online transactions, eliminating manual coordination and enabling seamless booking experiences. Combined with artificial intelligence, these capabilities can deliver personalized service recommendations through intuitive, accessible interfaces.

This paper presents BOOKMYCOOK, a comprehensive web-based event service booking platform that leverages React JS for a responsive frontend, Python Flask with SQLAlchemy for scalable backend operations, PostgreSQL for reliable data persistence, and JWT-based authentication with two-factor security. The platform supports chef, caterer, and decorator bookings across Tamil Nadu with AI-powered recommendations, real-time messaging, and integrated payment processing.

II. PROBLEM STATEMENT

The event services industry in Tamil Nadu operates through fragmented offline channels that fail to provide customers with unified access to professional service providers, making efficient service discovery and booking difficult. The core problems identified include:

- Fragmented market structure requiring customers to search multiple channels including personal referrals, social media groups, and offline directories to find suitable providers.
- Lack of transparency in pricing, service quality, and provider credentials, preventing informed decision-making and comparison.
- Absence of real-time availability management, leading to booking conflicts, scheduling errors, and provider unavailability.
- No centralized verification system for service providers, creating trust issues and uncertainty regarding credibility and reliability.
- Inefficient communication through multiple phone calls and messages, resulting in miscommunication and coordination failures.
- Cash-based transactions without digital payment options, creating security concerns and lacking documented transaction history.
- Limited geographic coverage with no platform specifically optimized for Tamil Nadu's unique market needs and preferences.

These deficiencies collectively result in frustrating booking experiences that fail to equip customers with the confidence and convenience required to successfully plan and execute memorable events.

III. EXISTING SYSTEMS AND LIMITATIONS

Current event service booking channels fall into word-of-mouth referrals, social media groups, and local business directories. Word-of-mouth referrals provide trusted recommendations from personal networks — reliable but severely limited in options and restricted to known providers within social circles.

Social media groups offer wider reach through Facebook and WhatsApp communities, but information is unstructured, verification is absent, and all coordination happens through scattered personal messages with no booking management. Local business directories list contact details and basic descriptions — accessible but often outdated, lacking reviews, and requiring separate phone calls for every inquiry.

Across all categories, none offer unified multi-service platforms, real-time availability checking, secure digital payments, verified provider profiles, AI-powered recommendations, or comprehensive booking management — capabilities that the proposed platform delivers.

IV. PROPOSED SYSTEM

A. Unified Service Marketplace

The platform introduces a centralized event service booking environment that aggregates professional chefs, catering services, and decorators across Tamil Nadu. Upon registration, users select their role as customer or service provider, which subsequently determines their access privileges and interface options. The system supports three distinct service categories with specialized features including cuisine type specifications, event type filtering, and location-based discovery across 44 cities.

B. Intelligent Service Discovery

Rather than relying on scattered offline channels, the platform provides dynamic, context-aware service recommendations through an AI-powered chatbot named "Cheffy." Each recommendation is constructed based on user preferences, budget constraints, and geographic location, ensuring results are relevant and aligned with specific event requirements. Advanced filtering capabilities including city, area, cuisine type, event type, and price range enable precise service discovery tailored to individual needs.

C. Secure Booking and Payment Integration

The platform implements a request-and-confirm booking model that ensures mutual convenience through provider availability verification before booking confirmation. Razorpay payment gateway integration supports multiple payment methods including cards, UPI, and net banking with automatic receipt generation. JWT-based authentication combined with two-factor authentication provides secure, stateless session management that protects user data through industry-standard token-based authorization and rate limiting.

D. Provider Dashboard and Administrative Control

Service providers access comprehensive management tools including service creation, availability calendar management, booking request handling, and earnings tracking through dedicated dashboard interfaces. Administrative features enable user verification, service approval workflows, location management for cities and areas, and analytics dashboards displaying platform metrics. Real-time messaging and notification systems facilitate seamless communication between customers and providers throughout the booking lifecycle.

V. ALGORITHM

The event service booking platform implements the following algorithmic pipeline:

Stage 1 — Secure Authentication: The user registers or logs in through a secure authentication module. JWT tokens are issued upon successful credential verification, with optional two-factor authentication using TOTP codes, and included in all subsequent API requests.

Stage 2 — Service Discovery and Filtering: Upon authentication, the system retrieves available services filtered by user-specified parameters including city, area, service type, cuisine types, event types, and price range. AI-powered recommendations through the Cheffy chatbot provide personalized suggestions based on user preferences and budget.

Stage 3 — Booking Request Creation: The customer submits a booking request containing event date, time, location, guest count, and special requirements. The system validates provider availability against stored calendar data and creates a pending booking record with a unique reference number.

Stage 4 — Provider Confirmation Workflow: The service provider receives a notification and reviews the booking request through their dashboard. The provider can accept or reject the booking based on availability, with status updates propagated in real-time to the customer through the messaging system.

Stage 5 — Payment Processing: Upon booking confirmation, the customer is redirected to Razorpay checkout. The backend creates a payment order, verifies the transaction signature upon completion, and updates the booking status with payment details and generated receipt.

Stage 6 — Service Completion and Review: After the event date, the system prompts the customer to submit a review with star rating and comments. The review is stored and aggregated to update the service's overall rating, while audit logs record all transactions for administrative analytics and reporting.

VI. TECHNOLOGIES USED

Frontend: React JS provides a component-driven reactive UI architecture with Tailwind CSS 4.x delivering utility-first, responsive styling through the @tailwindcss/postcss plugin. Vite serves as the modern build tool for optimized development and production builds.

Backend: Python Flask manages RESTful API routing, JWT authentication, rate limiting, and service orchestration. SQLAlchemy ORM provides type-safe database operations with relationship management and query optimization. UV accelerates dependency management with Rust-based performance.

Database: PostgreSQL stores user profiles, service listings, booking records, payment transactions, and audit logs with ACID compliance ensuring data integrity. SQLAlchemy models define the schema with JSON fields for flexible array storage of cuisine types, event types, and images.

AI Integration: The Cheffy chatbot provides AI-powered service recommendations through natural language processing, delivering personalized suggestions based on user preferences, budget constraints, and location requirements for enhanced service discovery.

Security: JSON Web Tokens (JWT) provide stateless, cryptographically signed authentication. Two-factor authentication using TOTP (pyotp) adds an additional security layer. bcrypt handles password hashing, while Fernet encryption protects sensitive data at rest.

Payment Integration: Razorpay API manages payment gateway operations including order creation, signature verification, and transaction tracking with support for cards, UPI, and net banking payment methods.

Development Tools: Visual Studio Code (IDE), Thunder Client/Postman (API testing), Ubuntu 22.04 / Windows 10 64-bit (OS), Git (Version Control), pytest (Testing Framework).

VII. EXPECTED OUTCOMES

Unified Service Access: Customers receive a centralized platform aggregating professional chefs, caterers, and decorators across Tamil Nadu, eliminating the need to search multiple fragmented channels and enabling comprehensive service comparison in a single interface.

Improved Booking Efficiency: The request-and-confirm booking model with real-time availability checking significantly reduces scheduling conflicts and miscommunication, ensuring seamless coordination between customers and service providers throughout the event planning process.

Secure Digital Transactions: Razorpay integration with JWT-based authentication and two-factor security provides safe, documented payment processing with automatic receipt generation, replacing risky cash-based transactions with transparent digital alternatives.

Provider Business Growth: Service providers gain access to a wider customer base through platform visibility, comprehensive dashboard tools for service and booking management, and earnings analytics that enable data-driven business decisions and operational improvements.

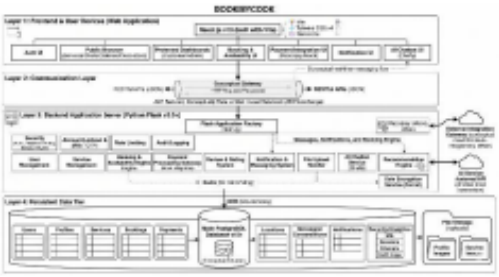
Comprehensive Administrative Control: PostgreSQL-backed data storage enables complete platform oversight with user verification workflows, service approval mechanisms, location management, and analytics dashboards. The modular architecture allows straightforward extension to new service categories, geographic regions, or features without disrupting existing functionality.

VIII. SYSTEM DESIGN OVERVIEW

The platform's architecture follows a clear client-server separation with dedicated payment and AI integration layers. The React JS frontend communicates with the Python Flask backend through JWT-secured RESTful APIs. The backend coordinates between PostgreSQL for data persistence, SQLAlchemy for ORM operations, and Razorpay for payment processing.

The service listing UI component structures the discovery interface into filterable cards, each displaying service images, provider details, pricing information, verified badges, rating summaries, and booking action buttons. Authentication follows a stateless flow: credential submission, JWT issuance with optional 2FA verification, token storage in sessionStorage, and Bearer-token validation by Flask-JWT-Extended middleware on every protected route.

Data models in PostgreSQL include User (profile, credentials, role, verification status), Service (title, description, type, pricing, location, images), Booking (reference, dates, status, amounts), Payment (transaction details, status), and Review (rating, comment, timestamps). Foreign key relationships and indexed queries on user ID, service ID, and booking timestamps ensure efficient retrieval of service listings, booking history, and analytics dashboards at scale.



IX. CONCLUSION

The BOOKMYCOOK event service booking platform addresses a significant and widely experienced challenge in Tamil Nadu's event services industry by delivering a unified, secure, and intelligent marketplace powered by modern web technologies and AI integration. By replacing fragmented offline channels with a centralized digital platform, the system ensures that every service discovery and booking experience is transparent, efficient, and reliable.

The integration of JWT-based secure authentication with two-factor protection, PostgreSQL-backed data persistence, and an intuitive React JS interface creates a cohesive system supporting seamless service discovery, secure digital payments, real-time communication, and comprehensive administrative control. The modular architecture ensures new service categories, geographic expansions, or additional features can be incorporated with minimal disruption.

Future enhancements will explore native mobile applications for broader accessibility, video calling for provider consultations, advanced analytics with predictive pricing, multi-language support for Tamil-speaking users, and integration with accounting software for provider business management. The platform establishes a strong foundation for the next generation of digital event service marketplaces.

REFERENCES

I. React Documentation, "React – A JavaScript library for building user interfaces," Online. Available: https://react.dev/. Accessed: Apr. 2025.

II. Pallets Projects, "Flask – Web Development, One Drop at a Time," Online. Available: https://flask.palletsprojects.com/. Accessed: Apr. 2025.

III. PostgreSQL Global Development Group, "PostgreSQL: The World's Most Advanced Open Source Relational Database," Online. Available: https://www.postgresql.org/docs/. Accessed: Apr. 2025.

IV. M. Bayer, "SQLAlchemy: The Database Toolkit for Python," Online. Available: https://www.sqlalchemy.org/. Accessed: Apr. 2025.

V. D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, Internet Engineering Task Force (IETF), 2012.

VI. J. Richer, "JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens," RFC 9068, Internet Engineering Task Force (IETF), 2022.

VII. Razorpay, "Razorpay Payment Gateway Documentation," Online. Available: https://razorpay.com/docs/. Accessed: Apr. 2025.

VIII. N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," Proceedings of the Annual Conference on USENIX, 1999.

IX. IETF, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, Internet Engineering Task Force, 2011.

X. Tailwind Labs, "Tailwind CSS – A Utility-First CSS Framework," Online. Available: https://tailwindcss.com/. Accessed: Apr. 2025.

XI. Astral, "UV – An Extremely Fast Python Package Manager," Online. Available: https://docs.astral.sh/uv/. Accessed: Apr. 2025.

XII. M. Stonebraker and L. A. Rowe, "The Design of POSTGRES," Proceedings of the ACM SIGMOD International Conference on Management of Data, 1986.

CHAPTER 13

CERTIFICATE

INTERNATIONAL CONFERENCE ON EMERGING INNOVATIONS IN SCIENCE, ENGINEERING AND TECHNOLOGY
(ICEISET - 2026)

CERTIFICATE
OF PARTICIPATION

This is to certify that (Dr./Mr./Ms.) Mohammed Aimman M

of University College of Engineering, BIT Campus has presented a paper titled

BOOK MY COOK – An Online chef Booking Service

in the International Conference on Emerging Innovations in Science, Engineering and Technology (ICEISET – 2026) - ISBN Number - 978-81-69495-15-8 on 30th April 2026, at Pencil Bitz, Coimbatore.

Conference Co-Chair | Conference Chair | General Chair